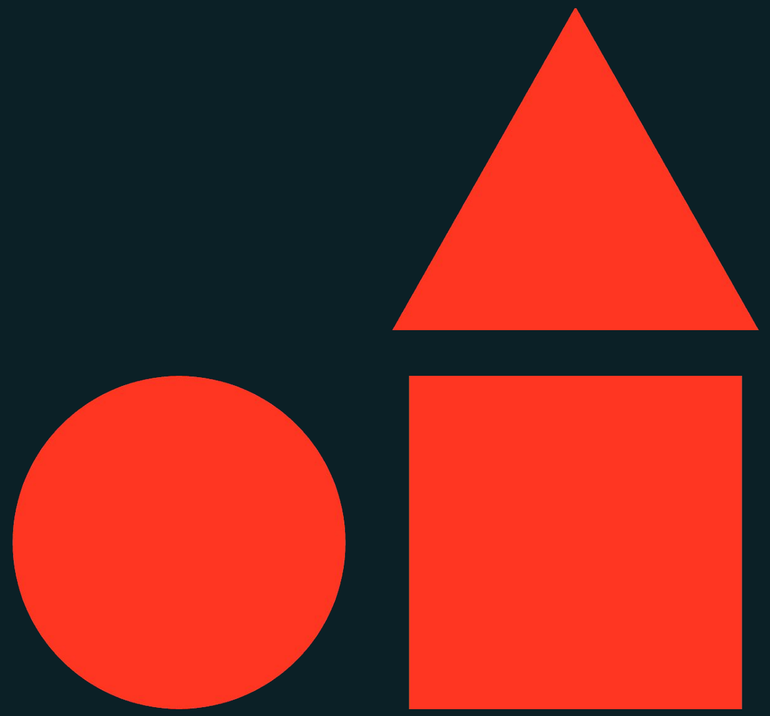
databricks

Machine Learning Operations



Databricks Academy

Course Learning Objectives

- Explain modern machine learning operations within the frameworks of DataOps, DevOps, and ModelOps.
- Relate MLOps activities to the features and tools available in Databricks, and explore their practical applications in the machine lifecycle.
- Design and implement basic machine learning operations, including setting up and executing a machine learning project on Databricks, following best practices and recommended tools.
- Detail Implementation and Monitoring capabilities of MLOps solutions on Databricks.

Prerequisites/Technical Considerations

Things to keep in mind before you work through this course

Prerequisites

1. Basic knowledge of traditional machine learning concepts
2. Beginner experience with traditional machine learning development on Databricks
3. Intermediate knowledge of Python for machine learning projects
4. **Recommended:** Beginner experience with basic DevOps concepts like CI/CD

Technical Considerations

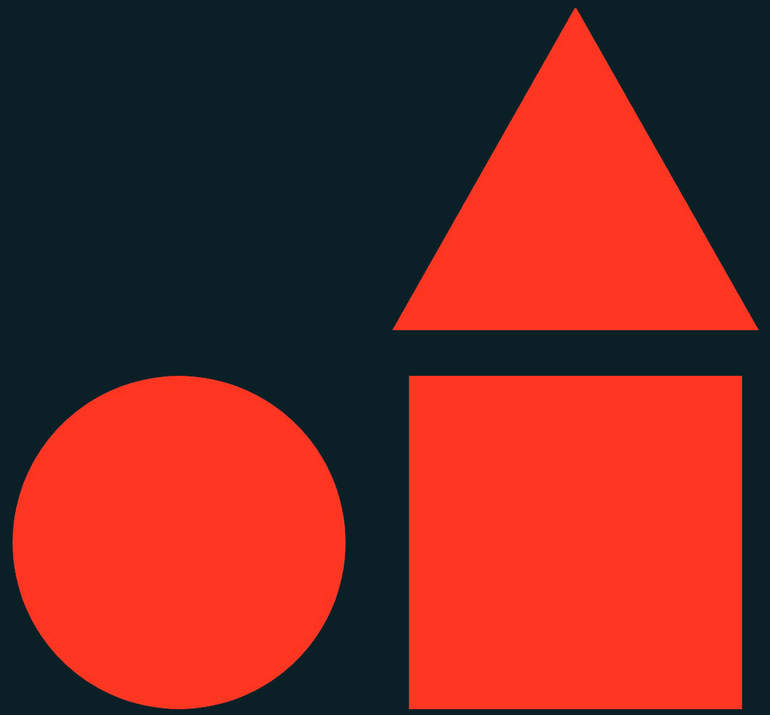
1. A cluster running on **DBR ML 16.3**
2. **Unity Catalog**, **Model Serving**, and **Lakehouse Monitoring** enabled workspace
3. CLI Authentication

AGENDA

1. Modern MLOps	Time	Lecture	DEMO	LAB
Defining MLOps	20 min	✔		
MLOps on Databricks	20 min 15 min 15 min	✔	✔	✔
02. Architecting MLOps Solutions				
Opinionated MLOps Principles	15 min	✔		
Recommended MLOps Architectures	25 min 15 min 15 min	✔	✔	✔
03. Monitoring MLOps Solutions				
Type of Model Monitoring	15 min	✔		
Monitoring in Machine Learning	15 min 25 min 30 min	✔	✔	✔

databricks

Modern MLOps



Machine Learning Operations

Learning Objectives

- Explain the significance of MLOps by integrating DataOps, DevOps, and ModelOps in modern machine learning.
- Identify and understand the components of DataOps, DevOps, and ModelOps within the context of machine learning.
- Describe Databricks' capabilities for handling tasks related to DataOps, DevOps, and ModelOps.
- Relate Databricks features and services to practical applications in DataOps, DevOps, and ModelOps tasks.

databricks

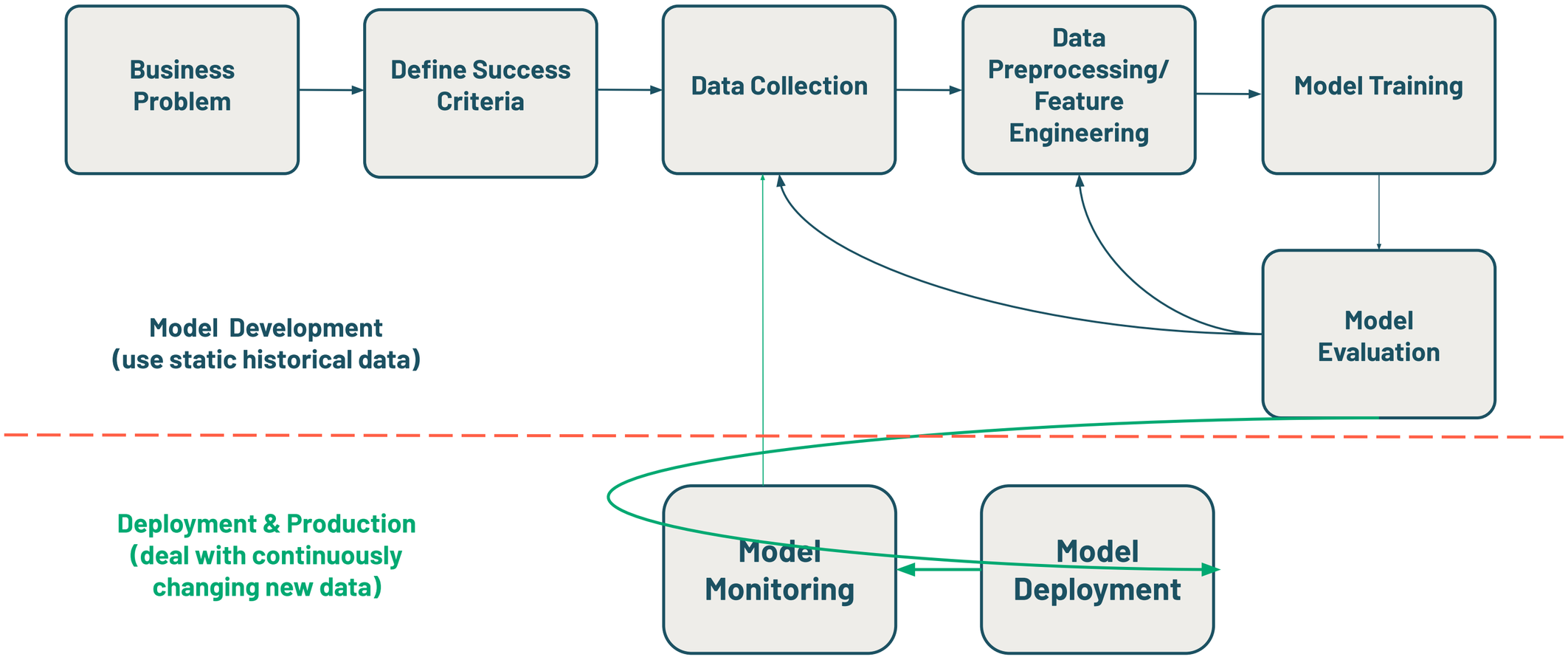
Modern MLOps

LECTURE

Defining MLOps

The Machine Learning Full Lifecycle

End to End Process from Business Problem to Deployment and Monitoring



Defining MLOps

An all-inclusive, holistic approach to managing ML systems

The set of practices, processes, and technologies for managing **data**, **code**, and **models** to improve performance stability and long-term efficiency in ML systems.

Understanding the Components of MLOps

MLOps components each address a specific part of ML projects

DataOps

A set of practices, processes, and technologies to organize and improve processes around data to increase speed, governance, quality, and collaboration.

DevOps

A set of practices, processes, and technologies to integrate and automate software development workflows.

ModelOps

A set of practices, processes, and technologies to organize and govern the lifecycle of machine learning and artificial intelligence models.

Responsibilities of MLOps Components

Key Functions of DataOps, DevOps, and ModelOps in MLOps

DataOps

- Optimized data **processing**
- Centralized data **discovery**, **management**, and **governance**
- Ensured data **quality**
- Traceable data **lineage** and **monitoring**

DevOps

- Machine learning is **code**
- Continuous integration and continuous deployment (**CI/CD)**
- **Version control** via Git
- Production-grade **workflows**
  - Orchestration
  - Automation

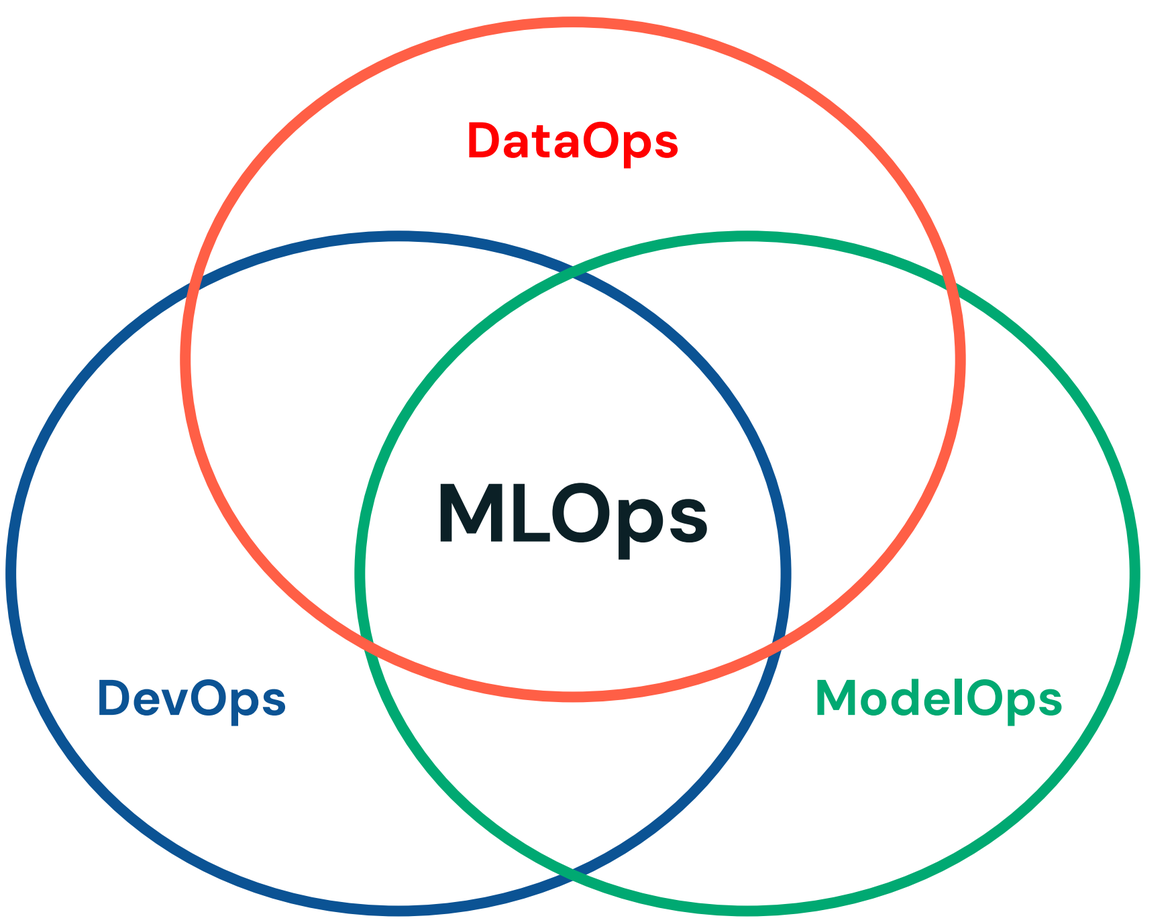
ModelOps

- Move beyond models as objects
- Treating model code as **software**
- Treating models as **data**
- Manage the **model lifecycle**

Comprehensive MLOps

Operationalizing the entire machine learning solution

The set of processes and automation for managing **data**, **code**, and **models** to improve performance stability and long-term efficiency in ML systems.



A Simple Example ML Project

Retail Recommendation System

1. A business owner defines a problem to be solved with a recommendation service.
2. A data scientist begins **exploring** **governed data** associated with the service.
3. A data scientist **develops** a **scalable** ML solution using using relevant **data** while **tracking the experiment**.
4. A machine learning engineer implements **CI/CD**, **automates** the ML solution, and establishes **model performance monitoring**; the data is written to a **production catalog**.

Why does MLOps matter?

Success depends on quality data and operations practices

- **Defining an effective strategy**
  - ML systems built on quality data
  - Streamlining the process of taking solutions to production
  - Operationalizing performance and effectiveness monitoring
- **So what?**
  - Time to realizing business value is accelerated
  - Reduction in manual oversight by high-value data science teams

Real-world Example

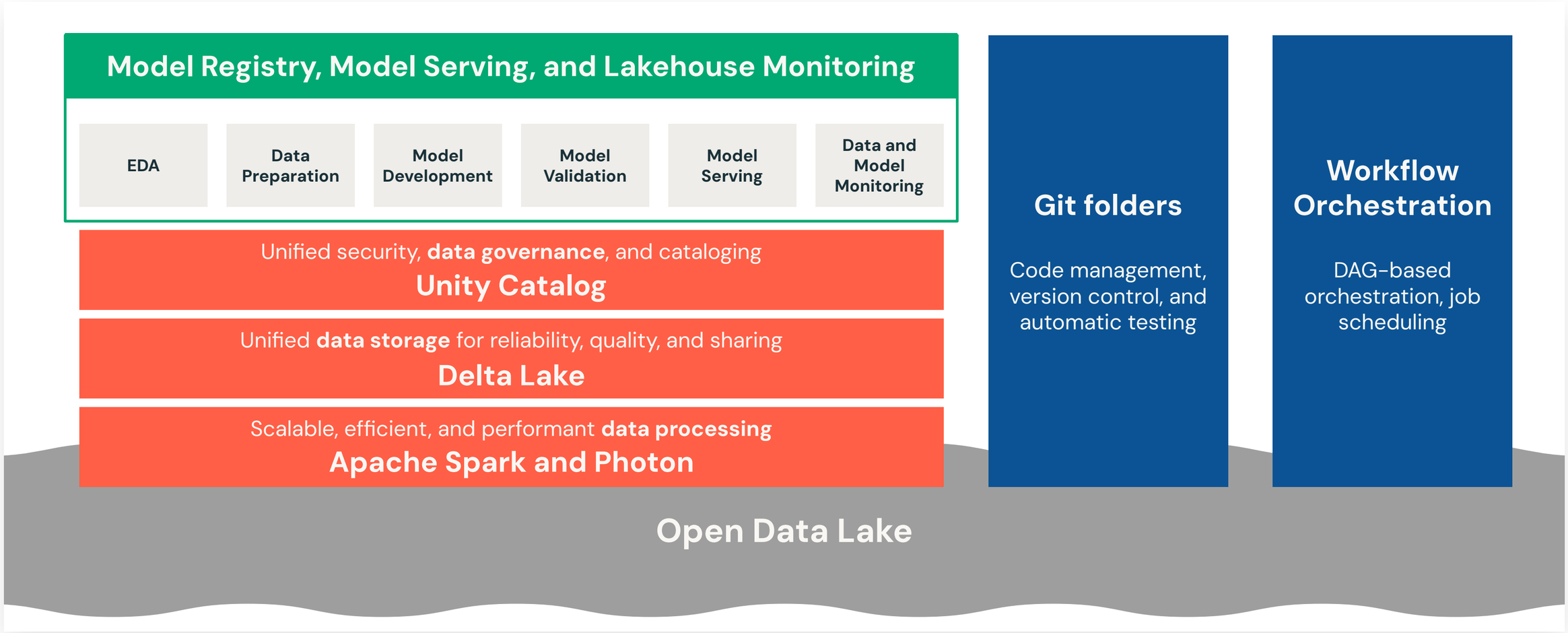
Databricks customer CareSource accelerated their model's development and deployment, resulting in a **self-service MLOps solution for data scientists** that reduced ML project time from 8 weeks to 3-4 weeks.

The CareSource team can extend this approach to other machine learning projects, realizing this benefit broadly.

Learn more about the work here.

A Single Platform for Modern MLOps

Combining DataOps, DevOps, and ModelOps solutions



databricks

Modern MLOps

LECTURE

MLOps on Databricks

DataOps tasks and tools in Databricks

The table lists common DataOps tasks and tools in Databricks:

Ingest & transform data	**Autoloader and Apache Spark***
Track data changes to including versioning & lineage	**Delta tables***
Build, manage, & monitor data processing pipelines	**Delta Live Tables***
Ensure data security & governance	**Unity Catalog***
Exploratory data analysis, dashboards, & general coding	**Databricks SQL, Dashboards, and Databricks notebooks**
Schedule data pipelines & Automate general workflows	**Databricks Workflows**
Create, store, manage, & discover features	**Databricks Feature Store***
Data monitoring	**Lakehouse Monitoring****

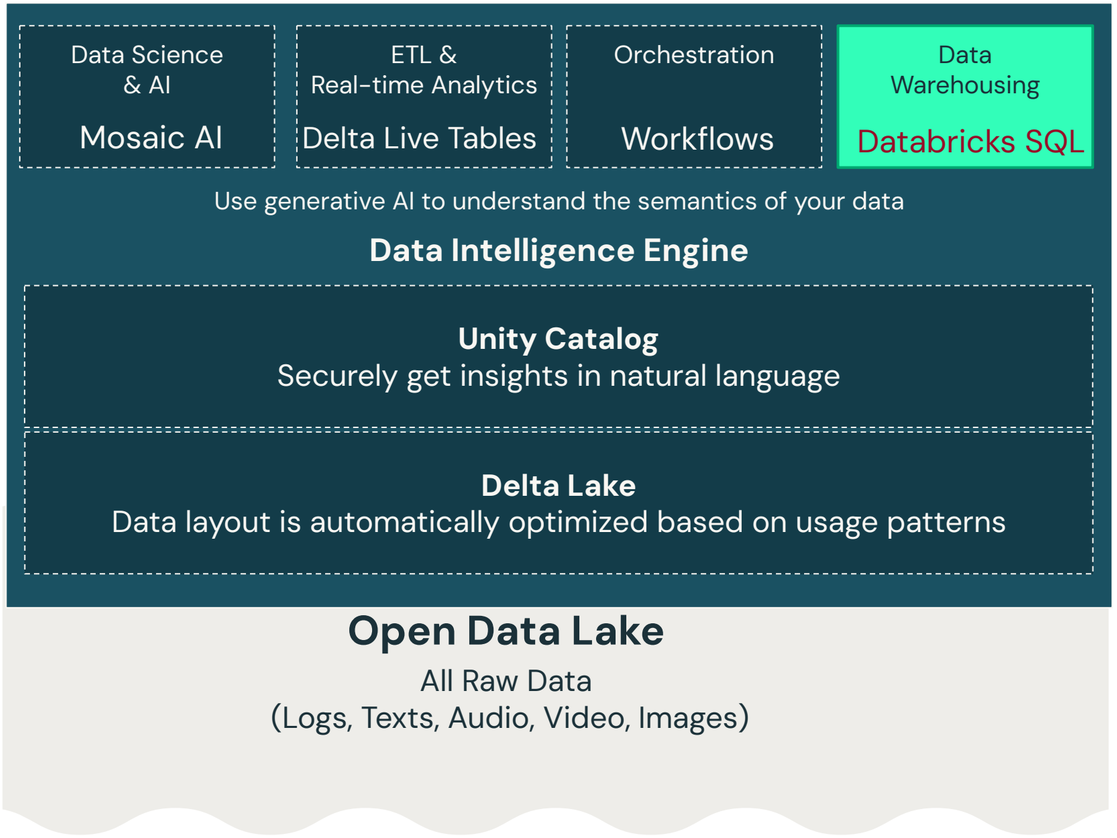
** Discussed in other associated machine learning courses.*

*** Covered later in the training*

Databricks SQL

Delivering analytics on the freshest data with data warehouse performance and data lake economics

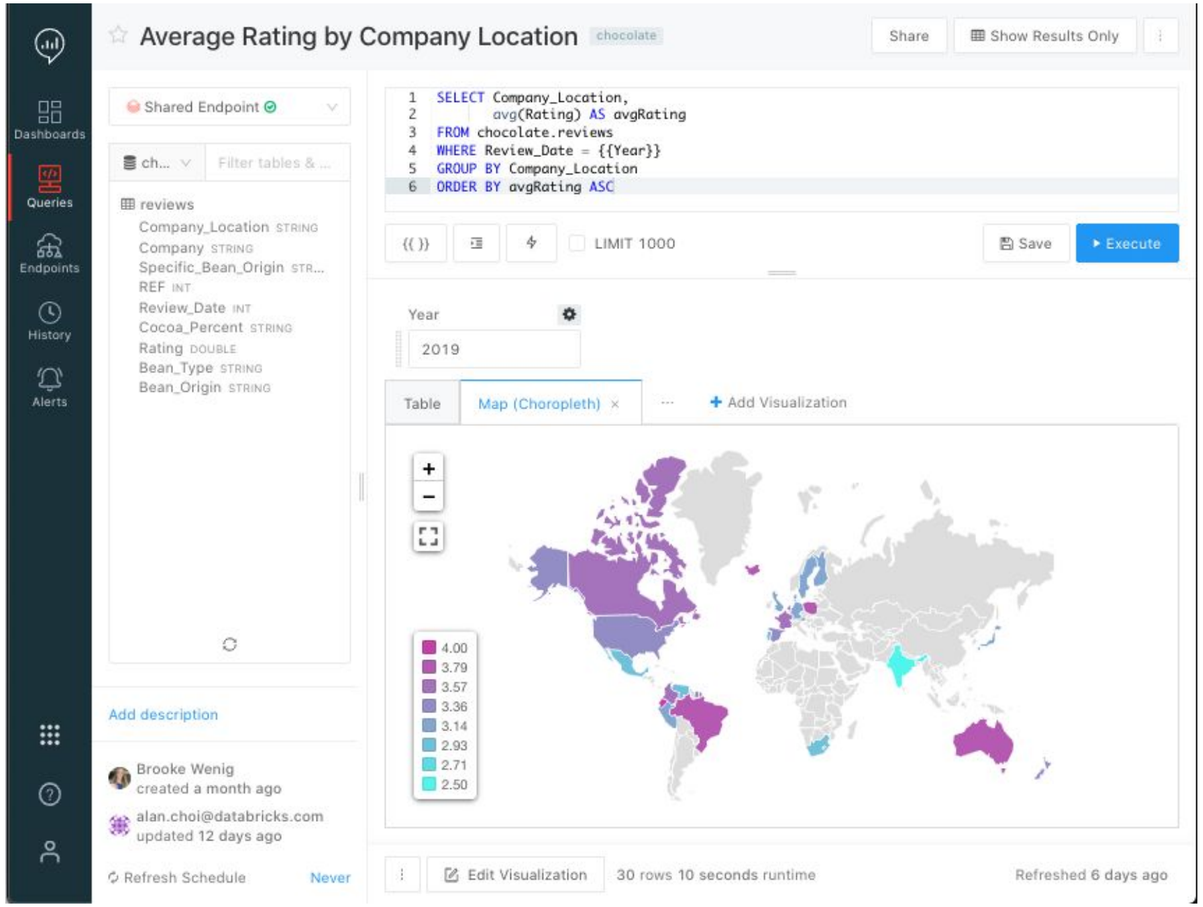
- Better price / performance than other cloud data warehouses
- Simplify discovery and sharing of new insights
- Connect to familiar BI tools, like Tableau or Power BI
- Simplified administration and governance



A new home for Data Analysts

- Enable data analysts to quickly **perform ad-hoc and exploratory data analysis**, with a new SQL query editor, visualizations and dashboards.

- **Automatic alerts can be triggered** for critical changes, allowing to respond to business needs faster.



DataOps Tasks and Tools in Databricks

The table lists common DataOps tasks and tools in Databricks:

Ingest & transform data	**Autoloader and Apache Spark***
Track data changes to including versioning & lineage	**Delta tables***
Build, manage, & monitor data processing pipelines	**Delta Live Tables***
Ensure data security & governance	**Unity Catalog***
Exploratory data analysis, dashboards, & general coding	**Databricks SQL, Dashboards, and Databricks notebooks**
Schedule data pipelines & Automate general workflows	**Databricks Workflows**
Create, store, manage, & discover features	**Databricks Feature Store***
Data monitoring	**Lakehouse Monitoring****

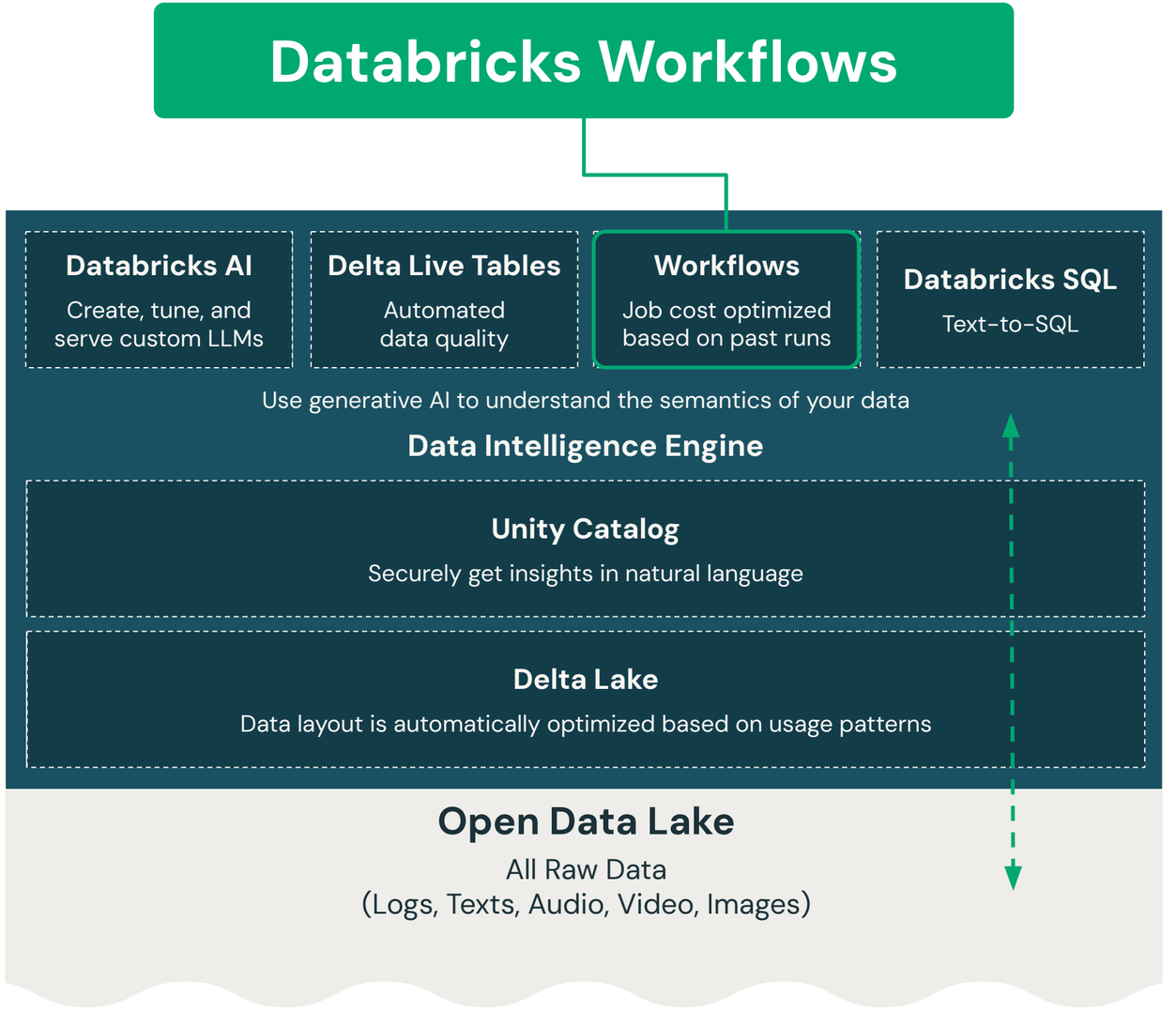
** Discussed in other associated machine learning courses.*

*** Covered later in the training*

Databricks Workflows

- Workflows is a **fully-managed cloud-based general-purpose task orchestration service** for the entire Lakehouse.

- Workflows is a service for **data engineers, data scientists and analysts** to build reliable data, analytics and AI workflows on any cloud.



Databricks Workflows

Databricks has two main task orchestration services:

Job Workflows

Delta Live Tables

• Execute jobs on a predefined schedule Series of interrelated tasks.	• Perform machine learning operations by running tasks within job frameworks like MLflow	• Implement a variety of tasks within a job • Using notebooks, JARS, Delta Live Tables pipelines, or Python, Scala, Spark submit, SQL and Java	• ETL processes • Compatible with batch and streaming inputs • Enforced data quality and consistency. • Tracking & logging of data transformation
Orchestration of Dependent Jobs	Machine Learning Tasks	Arbitrary Code, External API Call, Custom Tasks	Data Ingestion and Transformation

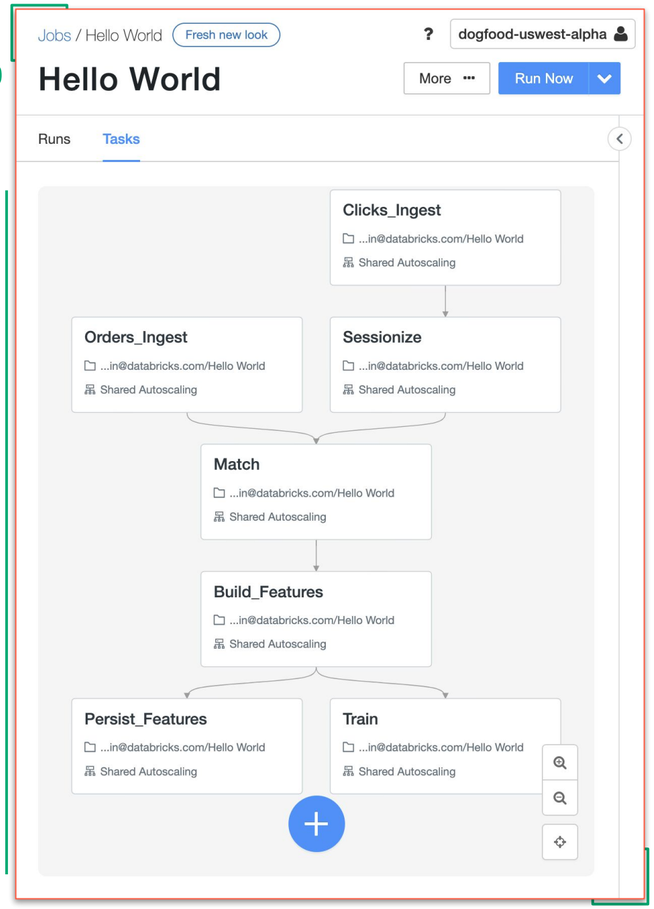
Note: DLT pipeline can be a task in a workflow

Workflows Jobs

Key Features

Workflow Job

DAG of tasks

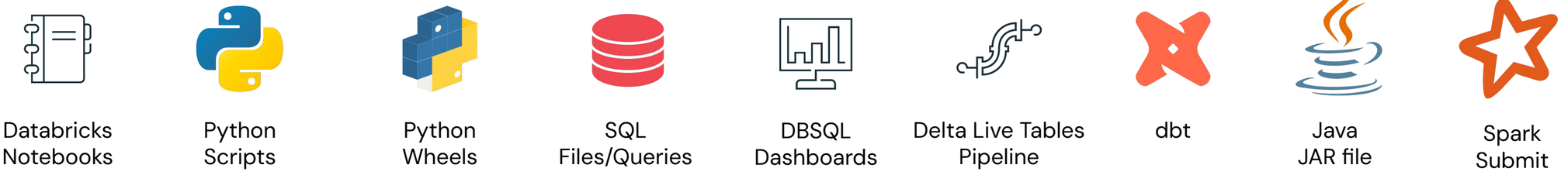


- **Easy creation, scheduling, and orchestration of your code with a DAG** (Directed Acyclic Graphs)
- Key features
  - **Simplicity:** Easy creation and monitoring in the UI
  - **Many Tasks** types suited to your workload
  - **Fully integrated** in Databricks platform, making inspecting results, debugging faster
  - **Reliability** of proven Databricks scheduler
  - **Observability** to easily monitor status

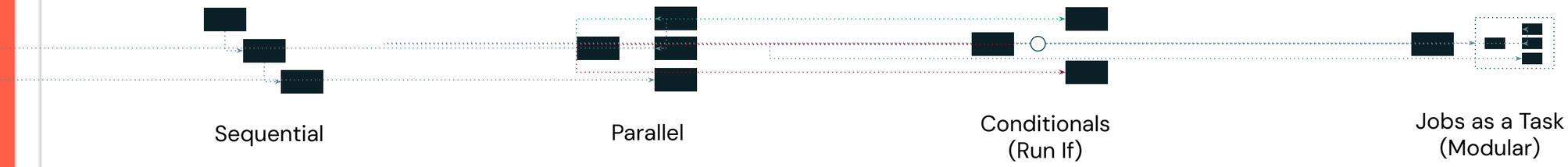
Building Blocks of Databricks Workflows Job

A unit of orchestration in Databricks Workflows is called a Job.

Jobs consist of one or more **Tasks**



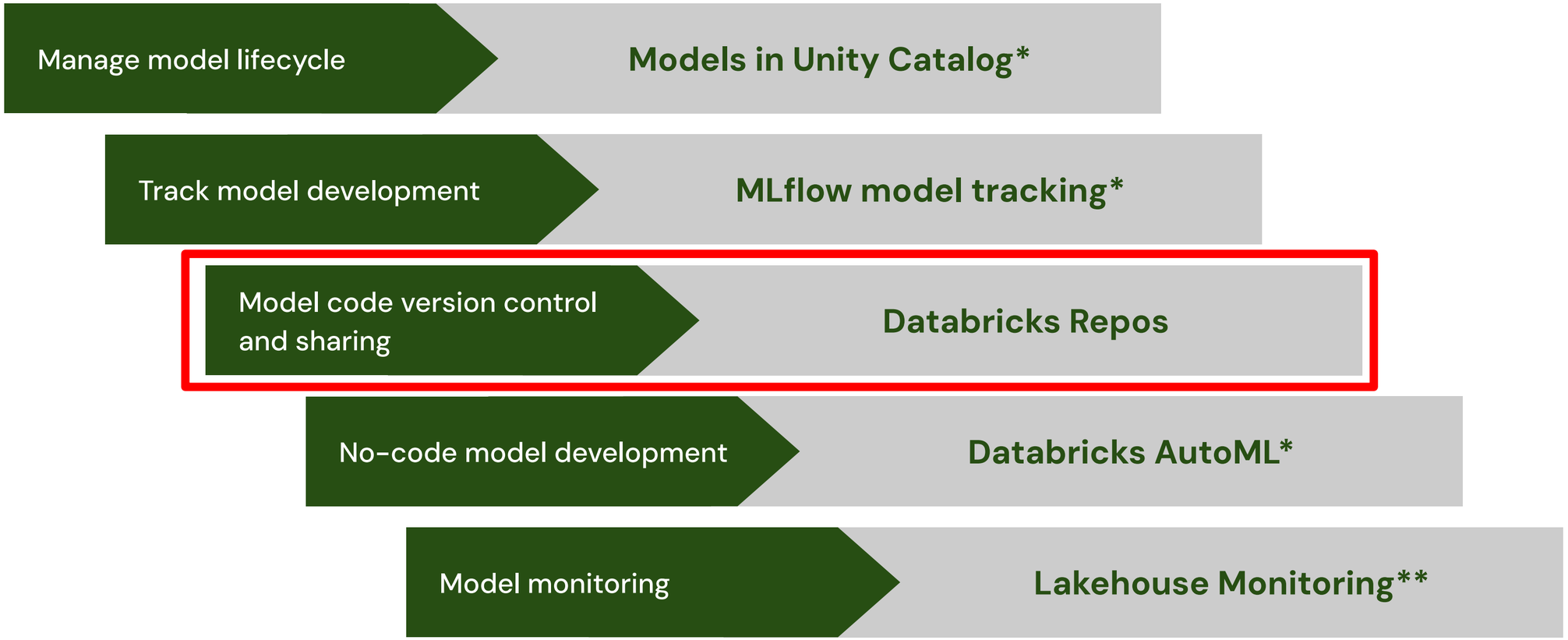
Control flows can be established between **Tasks.**



Jobs supports different **Triggers**

ModelOps Tasks and Tools in Databricks

The table lists common ModelOps tasks and tools provided by Databricks:

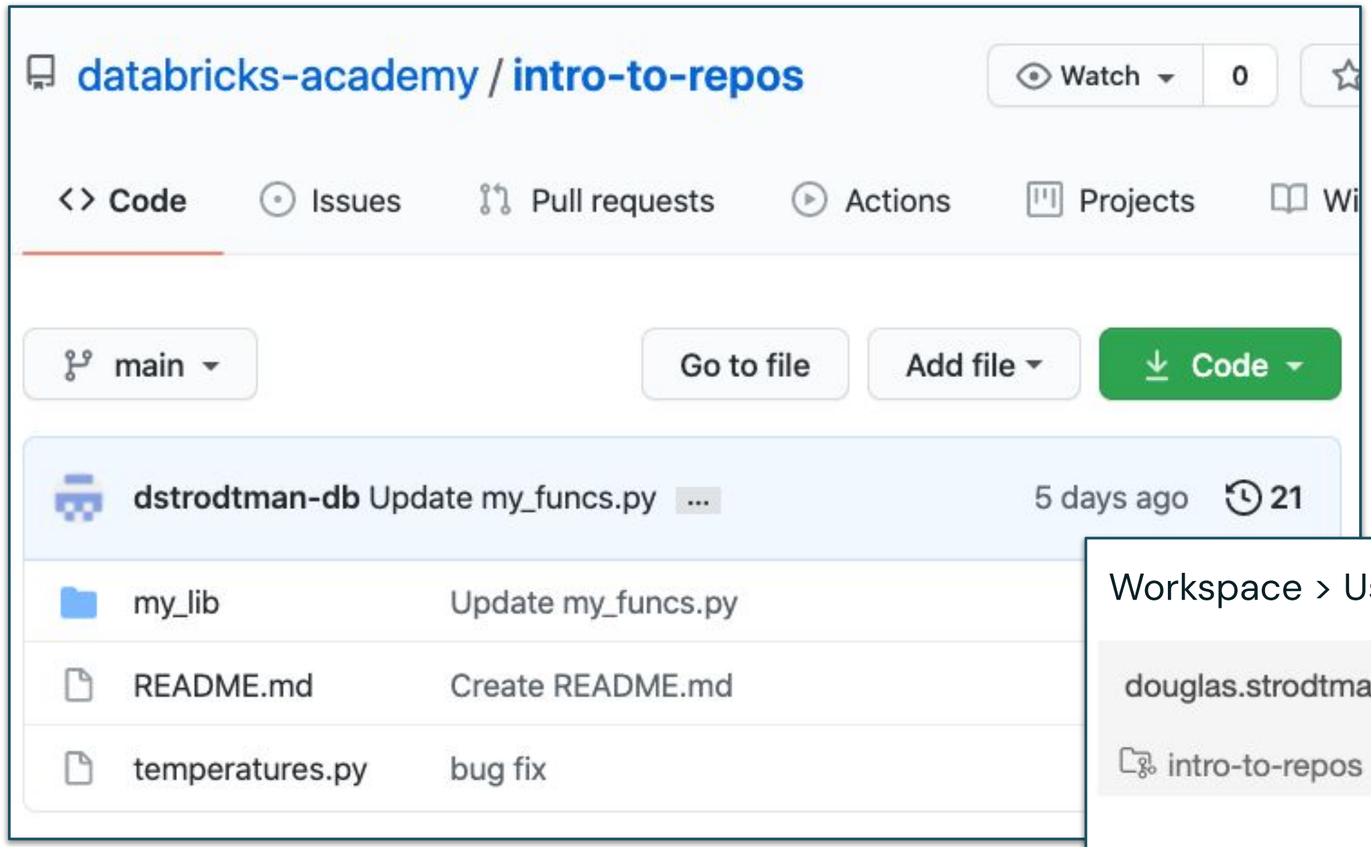


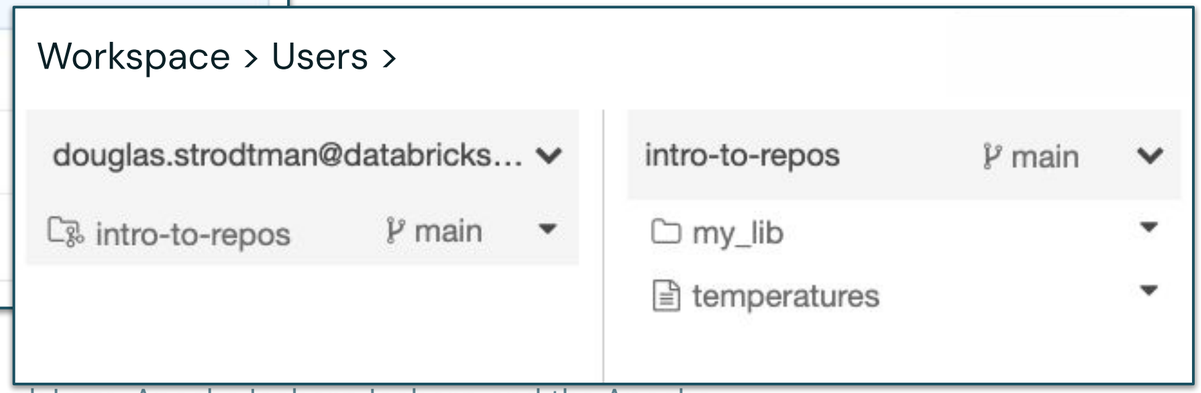
** Discussed in other associated machine learning courses.*

*** Covered later in the training*

Introduction to Git Folders

Git folders provide a **visual Git client** & **API** within Databricks, allowing users to **manage** code repositories, **collaborate**, and **integrate** with Git services.





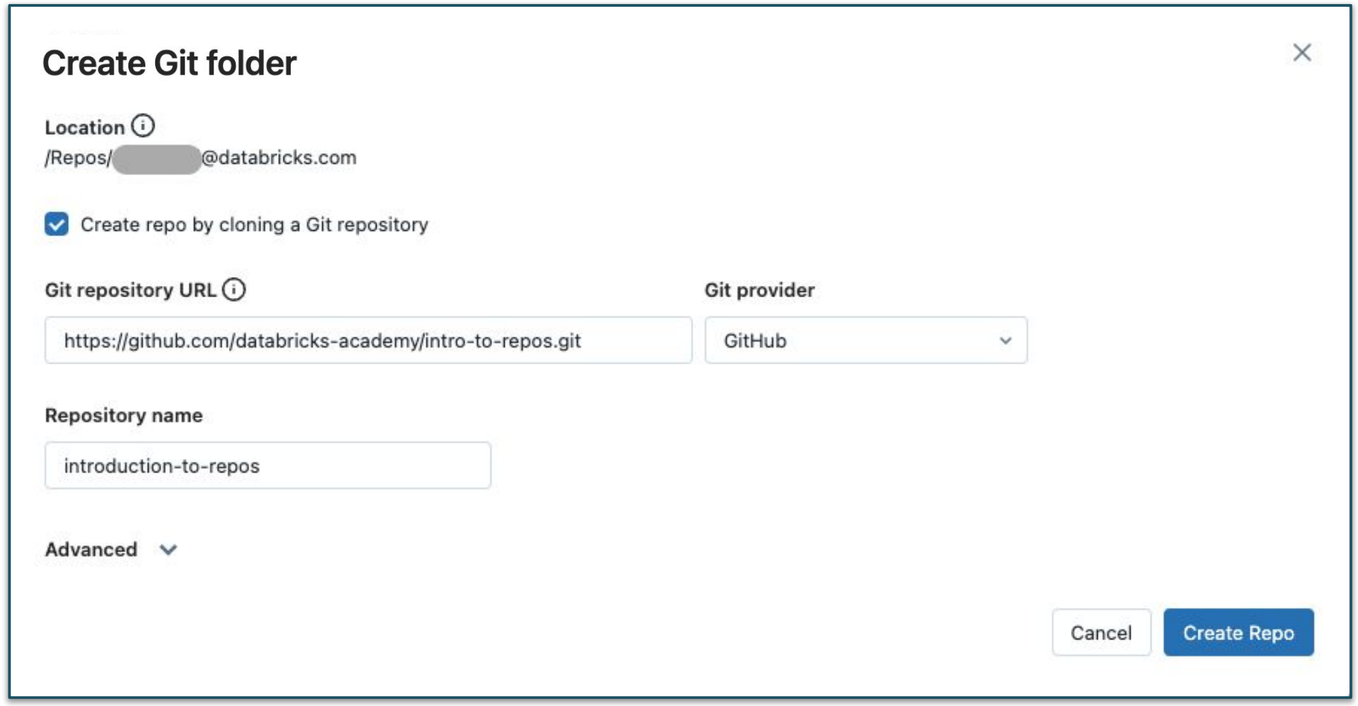
Key Features:

- Seamless Git integration.
- Collaborative coding environment.
- Simplified version control.

Git Folder Setup and Commands

Once setup easily manage and perform Git operations within Databricks

- From the Databricks UI common Git operations:
  - Clone
  - Checkout
  - Commit
  - Push
  - Pull
  - Branch management
- Uses Personal Access Token or equivalent to authenticate.



DevOps: Production and automation

The table lists common DevOps tasks and tools provided by Databricks:

Data and model lineage Access control and governance	**Unity Catalog***
Maintain a highly available low latency REST endpoint	**Mosaic AI Model Serving***
Automate and schedule workloads, from: ETL to ML	**Databricks workflows** ***Databricks also supports integrations with popular third party orchestrators like Airflow.***
Deployment infrastructure for inference and serving	**Asset Bundles, Databricks SDKs, Terraform provider, Databricks CLI**
Establish CI/CD Pipelines	**Databricks Asset Bundles, Azure DevOps, Jenkins, or GitHub Actions**
Monitoring, and Maintaining your Applications	**Lakehouse Monitoring****

** Discussed in other associated machine learning courses.*
*** Covered later in the training*

Overview Developer Tools

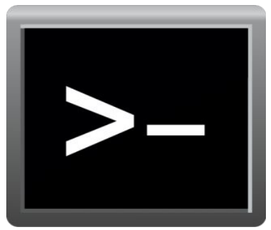
Databricks Asset Bundles (Recommended)

- Infrastructure as Code for Databricks resources and workflows

Databricks Asset Bundles

Databricks CLI

- Easy-to-use interface for automation from terminal, command prompt, or bash scripts.



Databricks SDKs

SDKs for:

- Python
- Java
- Go
- R

Databricks REST API

- Universal way to interact with Databricks workspaces: start/stop clusters, trigger jobs, manage models, update permissions, etc.

What is a DAB?

- **Databricks Asset Bundles or DABs** are a collection of Databricks **artifacts** [1] (e.g. jobs, ML models, DLT pipelines, and clusters) and **assets** [2] (e.g. Python files, notebooks, SQL queries, and dashboards).
- These DABs (aka **bundles**) are **configured through YAML files** and can be **co-versioned** in the same repository as the assets and artifacts referenced in the bundle.
- Using the **Databricks CLI** these bundles can be materialized across multiple workspaces **like dev / staging and production** enabling customers to integrate these into their **automation** and **CI/CD** processes

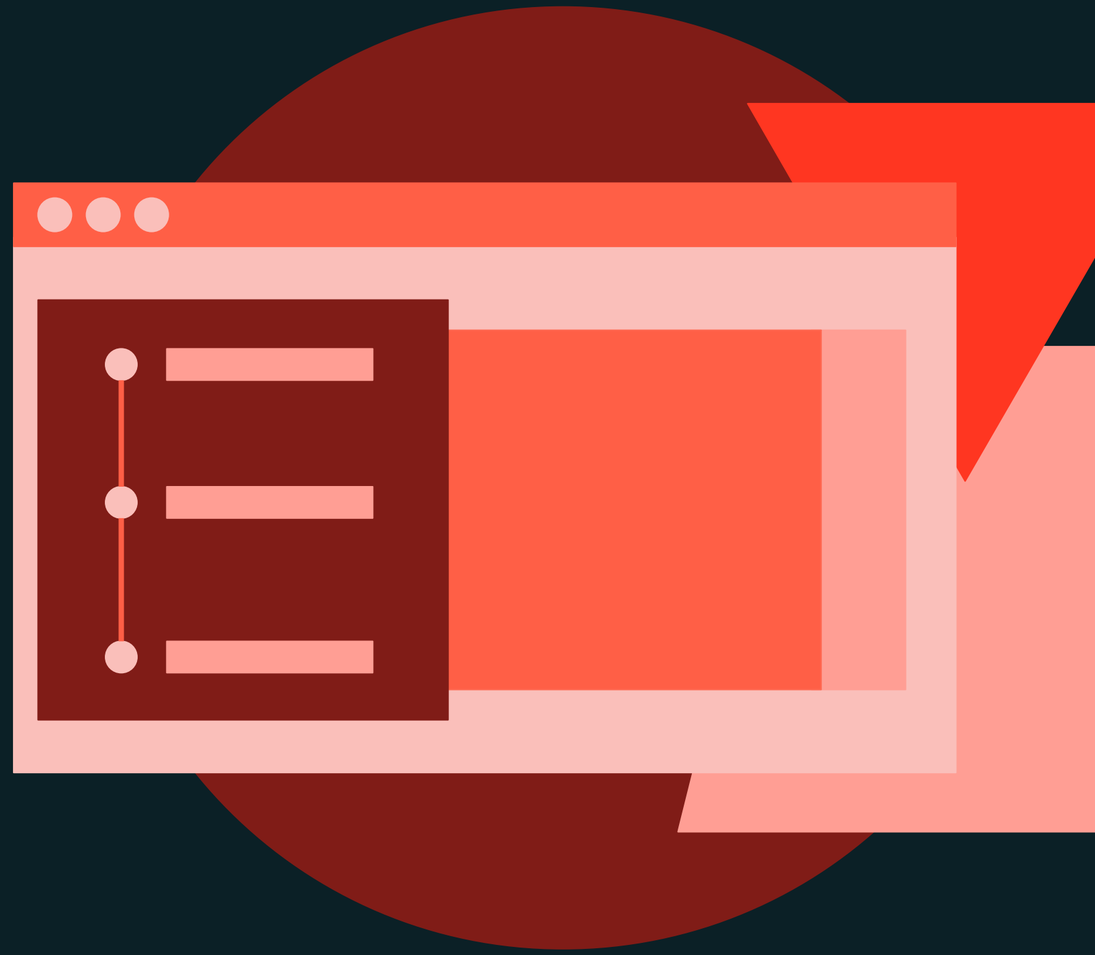
1 Assets are instantiations of sources that persist state.
2 Artifacts are file-like resources that exist on a workspace path that carry little or no state

databricks

Modern MLOps

DEMONSTRATION

Setting Up and Managing Workflow Jobs using UI



Demo

Outline

What we'll cover:

- Create and configure a Databricks Workflow job with multiple task Notebooks.
- Set dependencies and conditional paths between tasks.
- Enable email notifications for successful job runs.
- Manually trigger the workflow.
- Monitor the job's execution and completion.

databricks

Modern MLOps

LAB EXERCISE

Creating and Managing Workflow Jobs using UI

Lab

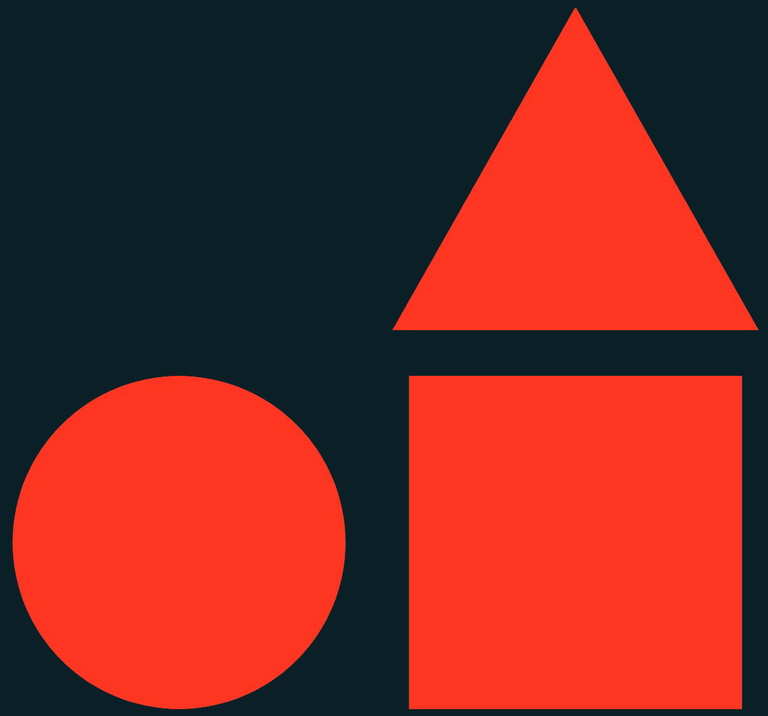
Outline

What you'll do:

- **Create and Configure a Workflow Job**
  - Setup multiple tasks using the UI.
- **Enable Email Notifications**
  - Configure notifications for job status updates.
- **Manually Trigger Deployment Workflow**
  - Initiate the job run manually.
- **Monitor Job Run**
  - Observe job execution and monitor the workflow.

databricks

Architecting MLOps Solutions



Machine Learning Operations

Learning objectives

Things you'll be able to do after completing **this module**

- Explain the importance of using the right MLOps architecture.
- Explain the reasoning behind the opinionated MLOps principles informing the recommended architecture of Databricks.
- Describe the Databricks-recommended MLOps architecture approach.
- Architect basic machine learning operations solutions for traditional machine learning applications based on Databricks-recommended best practices.

databricks

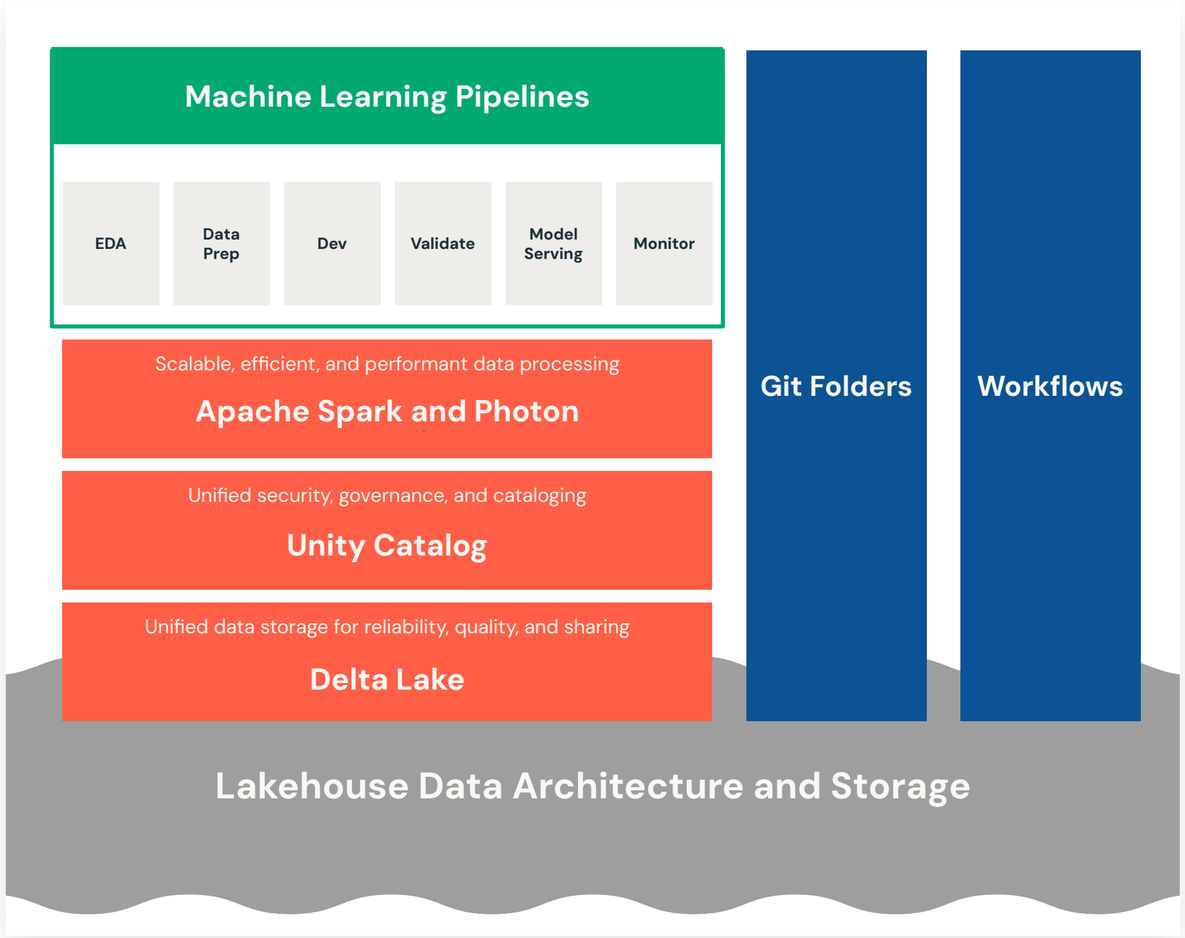
Architecting MLOps Solutions

LECTURE

Opinionated MLOps Principles

Guiding Principle

A **data-centric approach** to machine learning



- ML projects are made up of data pipelines
- Operationalizing ML solutions requires the connection of a variety of data pipelines
- Data pipelines require storage, governance, orchestration, etc.
- Aligned to the **Data Intelligence Platform vision**

Multi*-environment Semantics

Defining Development, Staging, and Production environments

Development

An environment where data scientists can **explore, experiment, and develop**

Staging

An environment where machine learning practitioners can **test** their solutions

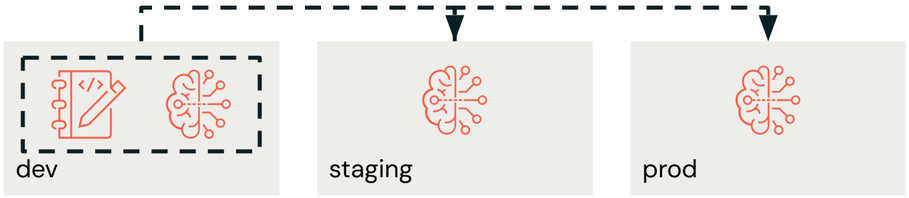
Production

An environment where machine learning engineers can **deploy and monitor** their solutions

Environment Separation

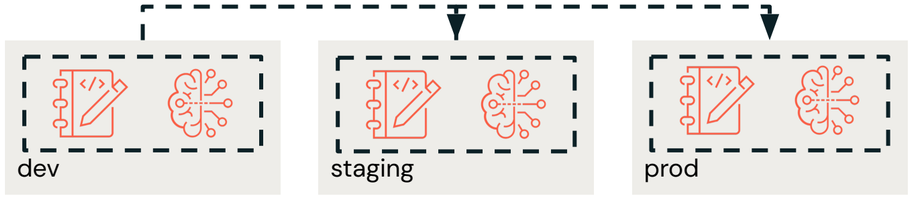
How many Databricks workspaces do we have?

Direct Separation



- Completed separate Databricks workspaces for each environment
- Simpler environments
- Scales well to multiple projects

Indirect Separation

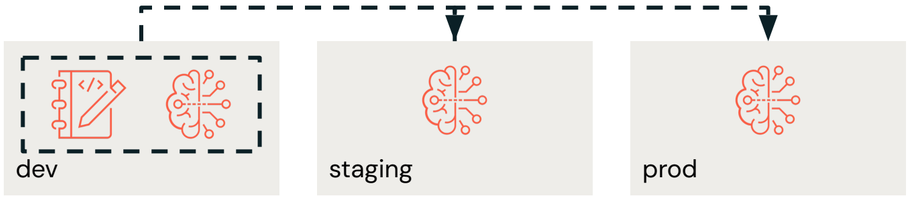


- One Databricks workspace with enforced separation
- Simpler overall infrastructure requiring less permission required
- Complex individual environment
- Doesn’t scale well to multiple projects

Deployment Patterns

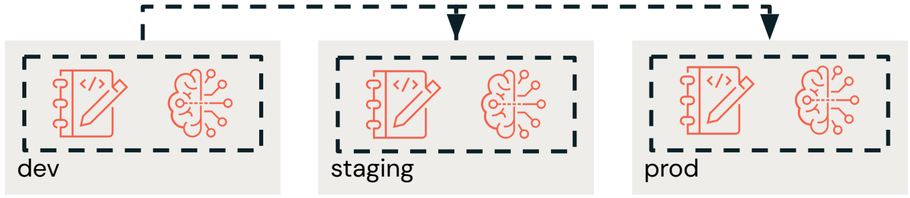
Moving from **Deploy Model** to **Deploy Code**

Deploy Model



- Model is trained in development environment
- Model artifact is moved from staging through production
- Separate process needed for other code (inference, monitoring, operational pipelines)

Deploy Code (recommended)



- Code is developed in the development environment
- Code is tested in the staging environment
- Code is deployed in the production environment
- Training pipeline is run in each environment, model is deployed in production

databricks

Architecting MLOps Solutions

LECTURE

Recommended MLOps Architectures

Importance of MLOps Architecture

Setting an ML project up for success starts with architecture

Simplicity

When ML projects are well architected, the downstream management, maintenance, and monitoring of the project is simplified.

Efficiency

When ML projects are well architected, processes around the project become more efficient.

Scalability

When ML projects are well architected, they can easily be scaled to adapt to changing requirements for infrastructure and compute.

Collaboration

When ML projects are well architected, it's easy for different users and different types of users to collaborate effectively.

Dimensions of Architecture

Differentiating initial organization/setup and ongoing workflows

Infrastructure

The organization, governance, and setup of environments, data, compute and other resources.

- Set up one time (per project or per team/organization)
- Crucial to downstream success of project(s)

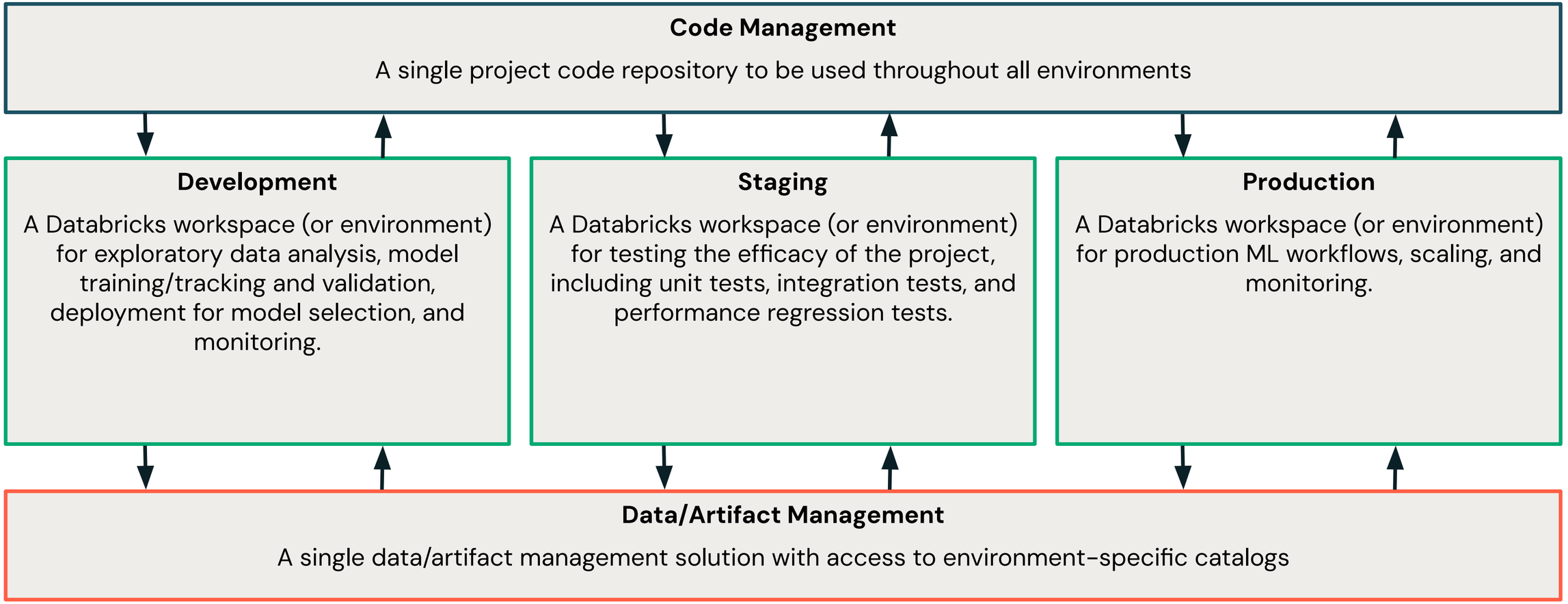
Workflow

The processes that ML practitioners follow within a defined architecture to achieve success on an ML project.

- Repeatable, fluid processes specific to a project
- Aligned to organizational best practices

Recommended MLOps Architecture

A high-level view of code, data, and ML environments



MLOps Solution

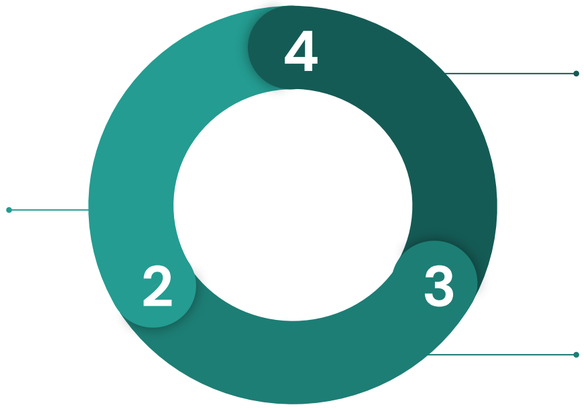
Infrastructure through production

1
Infrastructure Setup
*Optional**

Organization and set up of infrastructure for a machine learning project

Development

Developing the EDA and ML Pipelines of the ML solution.



Production

Setting up the deployment and monitoring of the production-grade ML solution.

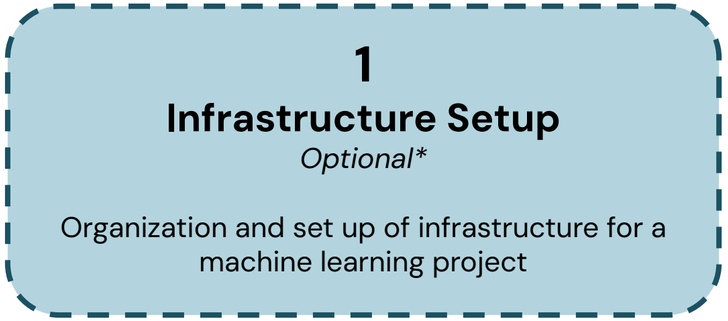
Staging

Establishing the automated testing setup for the ML solution.

Infrastructure Setup

Getting set up for a machine learning project

- **Who:** Architect or Engineer
- **How often:** Set up once
- **What:**
  - A Unity Catalog metastore
  - One or three Databricks workspaces
  - Three data catalogs: dev, staging, prod
  - Command line environment
  - *MLOps Stacks* project (per project)
  - Git repository (per project)
- **How:**
  - Manual setup/Terraform
  - MLOps Stacks



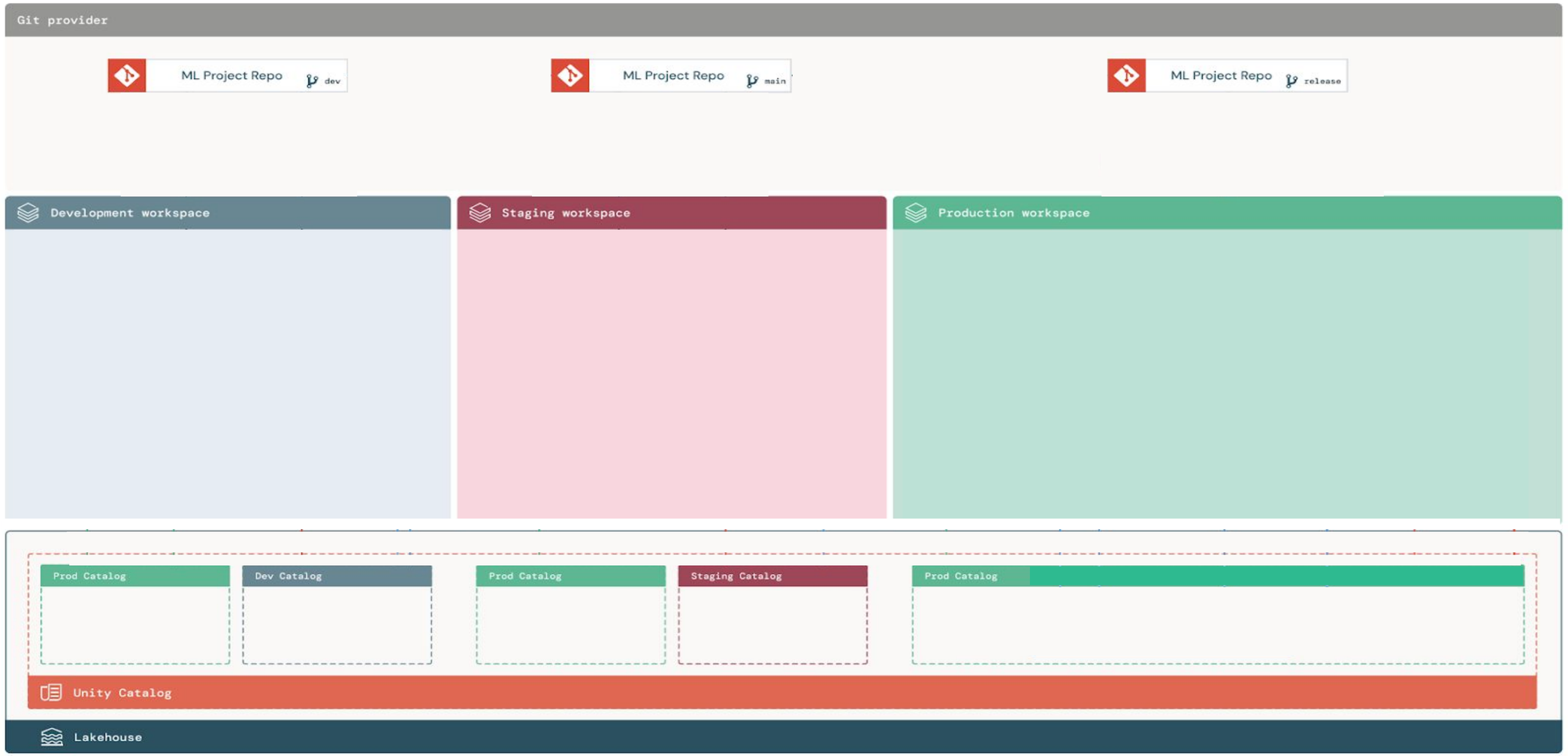
Infrastructure Setup Tasks

Using our DataOps + DevOps + ModelOps mental model

- Make Infrastructure Decisions:
  - Number of **workspaces**
  - Use existing vs. new infrastructure
  - Select **CI/CD** tooling
- Create Databricks Environment:
  - **Unity Catalog Metastore**
  - 1 or 3 workspaces
  - **Unity Catalogs** (dev, test, staging, prod)
  - **Service principal permissions**
- Optimize for Efficiency:
  - **Project templates** with guardrails and best practices configured.
  - **Git repository** creation and connection
  - Configure Databricks **CLI and IDE**
    - *e.g., VS Code extension*
- Additional Considerations:
  - **Monitoring and logging**
  - **Backup and recovery**
  - Security best practices
  - Network configuration

Deep Dive: Infrastructure

A closer look at the Infrastructure stage



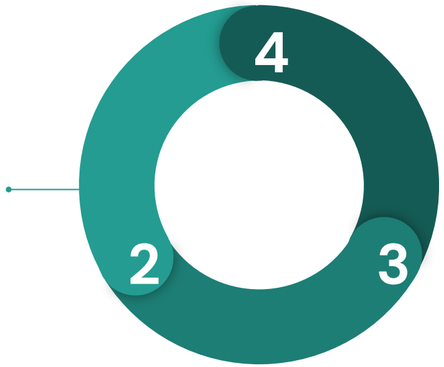
Development

Developing a machine learning project

- **Who:** Data Scientist
- **How often:**
  - Initial solution development
  - Solution updates
- **What:**
  - EDA
  - ML Development
  - ML Validation and Deployment
  - Monitoring solution
- **How:**
  - Develop ML pipelines within project architecture by editing an MLOps Stacks project

Development

Developing the EDA and ML Pipelines of the ML solution.



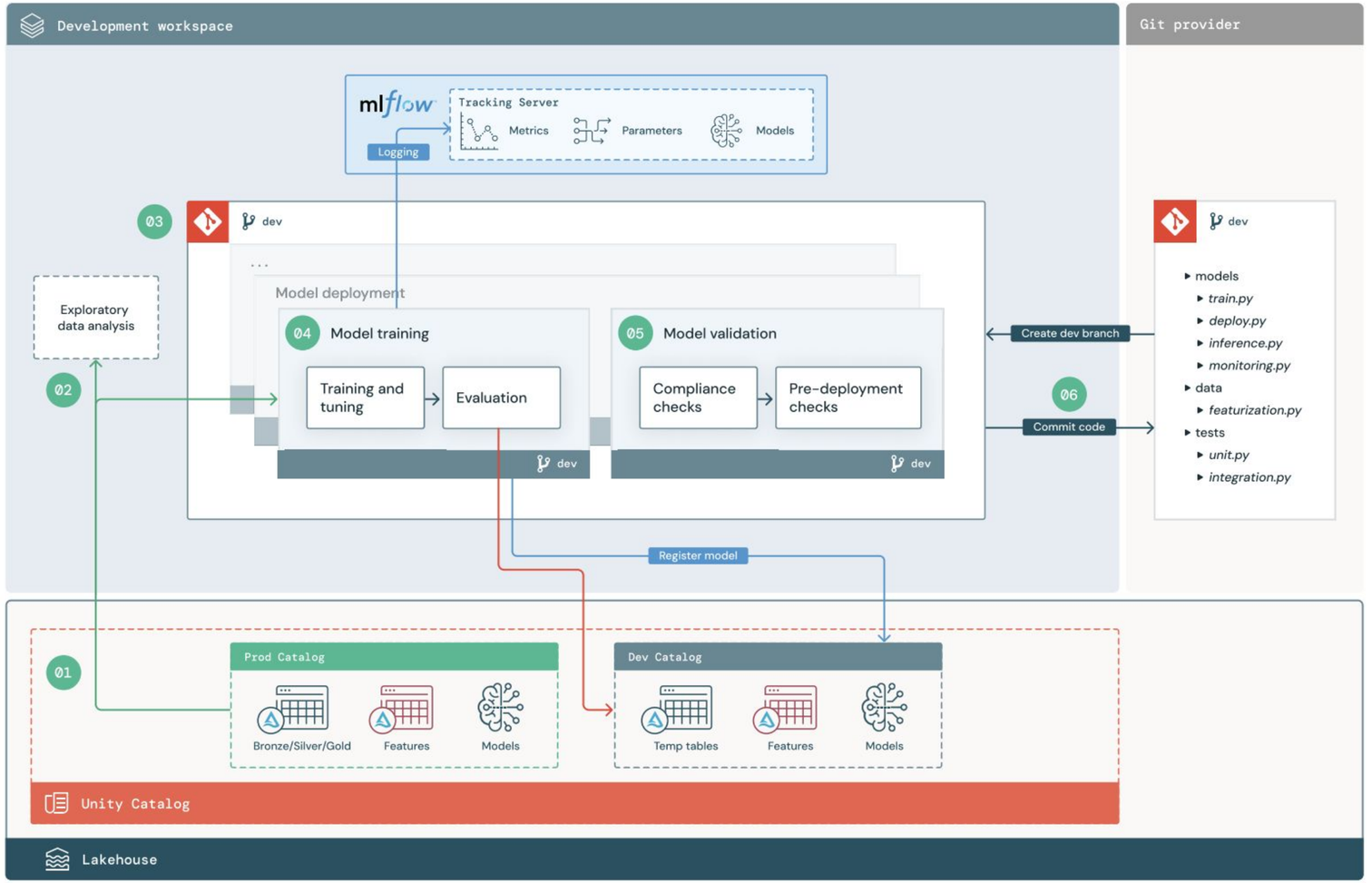
Development Tasks

Using our **DataOps** + **DevOps** + **ModelOps** mental model

- Make Changes to Code:
  - Add and update **code**
  - Use **project templates** (DAB, MLOps Stacks, etc.)
  - Write **notebooks** and **scripts**
  - Create queries and alerts
  - Commit and pull code changes
- Validate Data and Code:
  - Ensure correct setup before deployment
  - Validate using **DLT, Asset Bundles**, etc.
  - Confirm **data format compliance**
  - Implement **data quality checks**
  - Review for security and compliance
- Deploy Solution:
  - Deploy all **jobs** to ensure successful execution
  - Use **CLI, scripts**, and **automation** for consistent deployments across environments
  - Establish **rollback strategies**
- Additional Considerations:
  - Implement **version control**
  - Optimize **deployment workflow**
  - Ensure scalability and performance **tuning**
  - Set up automated **notifications**

Deep Dive: Development

A closer look at the Development stage



Legend

Workflow	Reads	Git repo
Job/Workflow task	Writes	Git branch
Pipeline step	MLflow API	Registered model

Staging

Testing a machine learning project

- **Who:** ML Engineer
- **How often:**
  - Set up and run every time a change is made in Development
  - Run every time a model is refreshed
- **What:**
  - Project merge
  - Project code testing
- **How:**
  - Centralized Git repository
  - Automated CI/CD infrastructure tools

Staging

Establishing the automated testing setup for the ML solution.

Staging Tasks

Using our DataOps + DevOps + ModelOps mental model

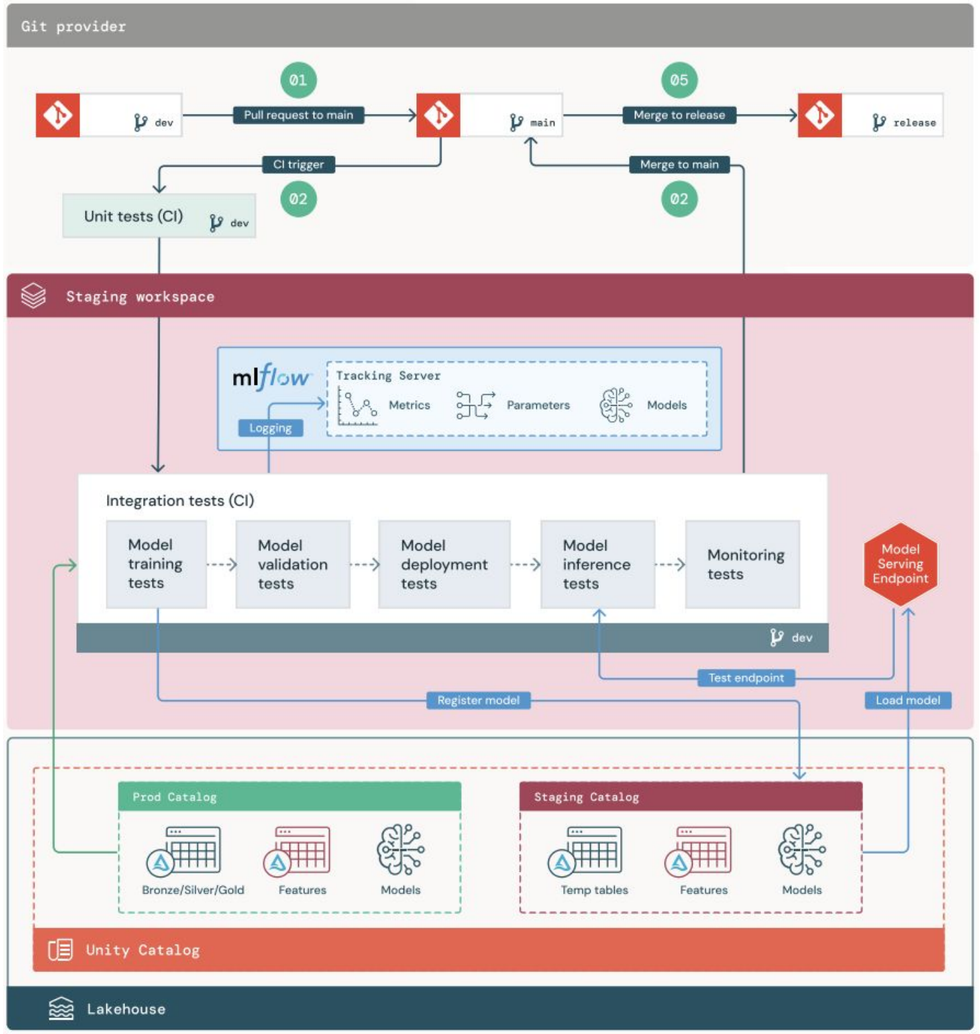
- Review CI/CD Workflows and Tests:
  - Examine existing **CI/CD** workflows
  - Add/change **tests** as needed
  - Ensure workflow alignment with project
- Create Pull Request to Run Tests:
  - Establish a **trigger**
    - e.g., pull request to merge dev branch into main branch
  - **Run Tests**:
    - Check for conflicts
    - Validate CI/CD setup
    - Validate the project
    - Unit, integration, and stress tests

- Analyze Test Results
  - If pass all tests and get approved,
    - merge into the main branch
  - If tests fail,
    - return to the development stage
  - Review test **reports and logs** for insights

- Additional Considerations:
  - Implement automated notifications for test results
  - Monitor staging environment performance
  - Ensure **data integrity** and consistency during staging

Deep Dive: Staging

A closer look at the Staging stage



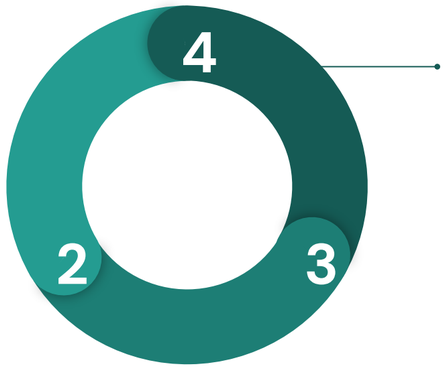
Legend

Workflow	Reads	Git repo
Job/Workflow task	Writes	Git branch
Pipeline step	MLflow API	Registered model

Production

Deploying and monitoring a machine learning project

- **Who:** ML Engineer
- **How often:**
  - When changes are made and tests are passed
  - When the model needs refreshed
- **What:**
  - Automated run/deployment of solution
  - Monitoring of solution
- **How:**
  - Centralized Git repository
  - MLOps Stacks project



Production

Setting up the deployment and monitoring of the production-grade ML solution.

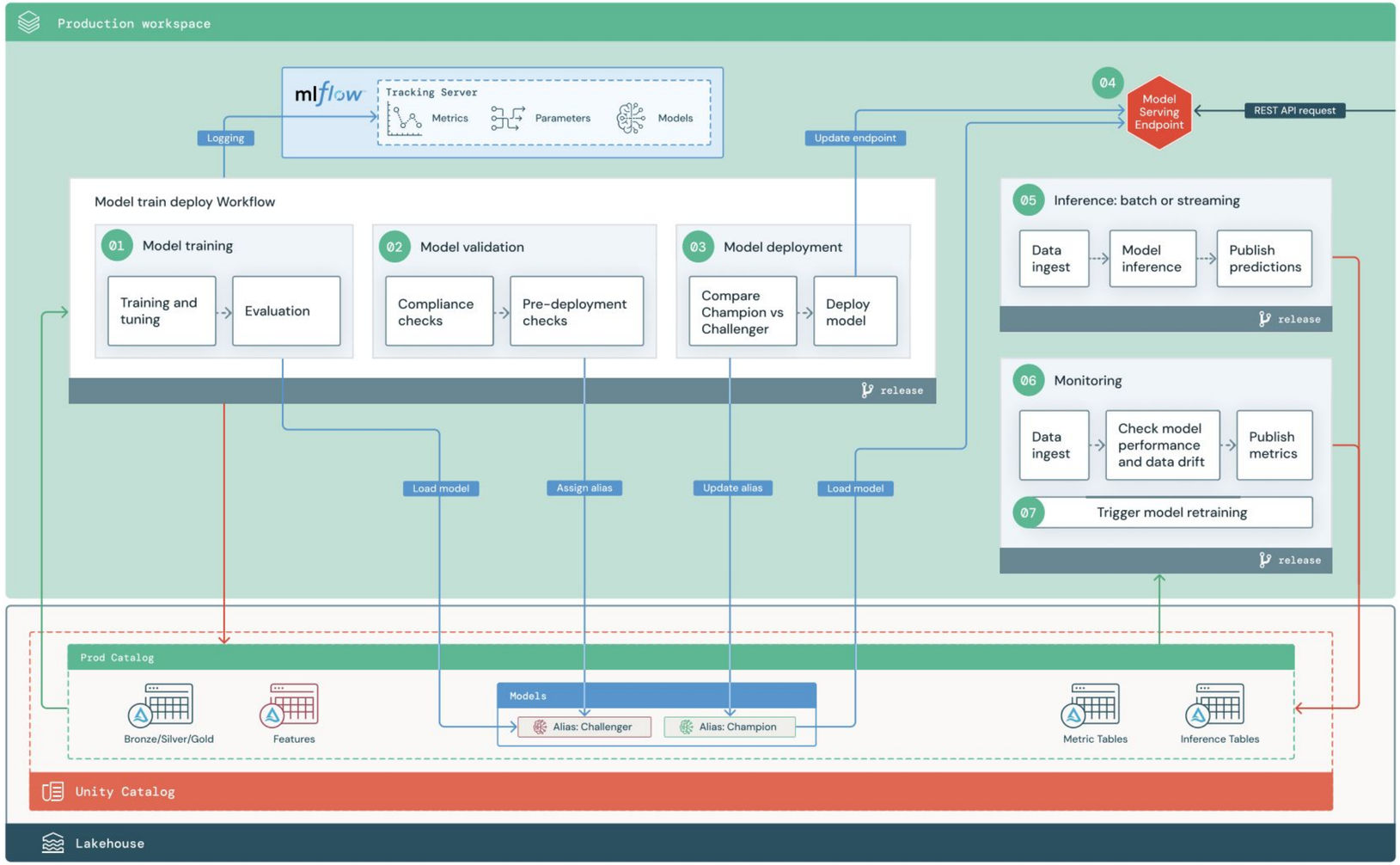
Production Tasks

Using our **DataOps** + **DevOps** + **ModelOps** mental model

- Create/Merge to Release Branch:
  - Set up release branch
  - Include deployment **triggers**
  - Merge updates into release branch
- Deploy Solution:
  - Automatic **deployment triggered** by release branch updates
  - Deploy components:
    - **Project code**
    - **Data**, **Model**, and **Monitoring** workflows
    - **Compute resources**
- Monitor
  - Track **system performance**
  - Monitor **deployment health**
  - Real-time **alerting** and notifications
  - Log analysis for error detection
  - **Data** and **model** drift detection
- Additional Considerations:
  - Optimize **resource utilization**
  - Ensure compliance with security policies
  - Conduct post-deployment **reviews**
  - Implement **incident response** protocols
  - Set up **automated retraining pipelines**

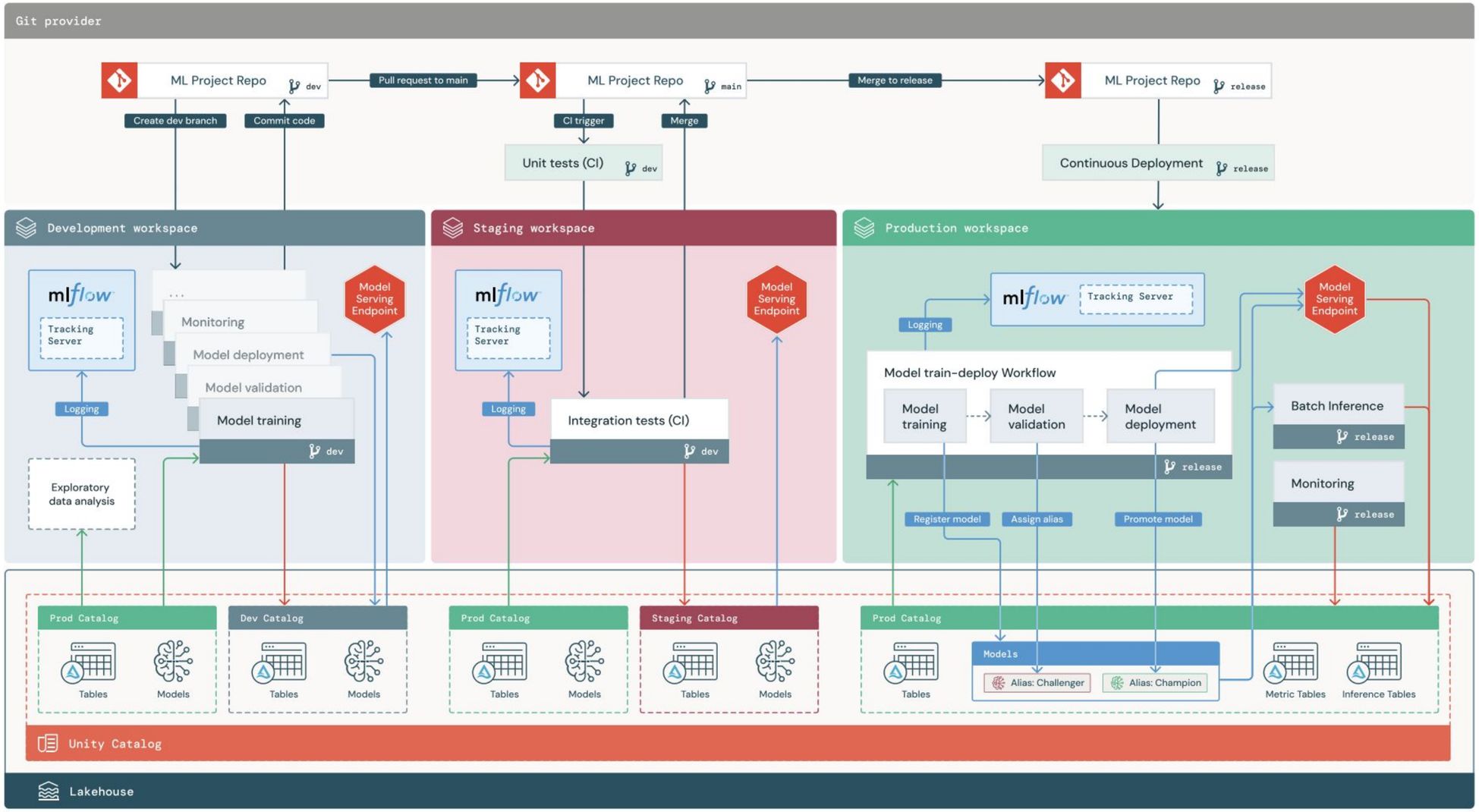
Deep Dive: Production

A closer look at the Production stage



Legend

Complete MLOps Architecture

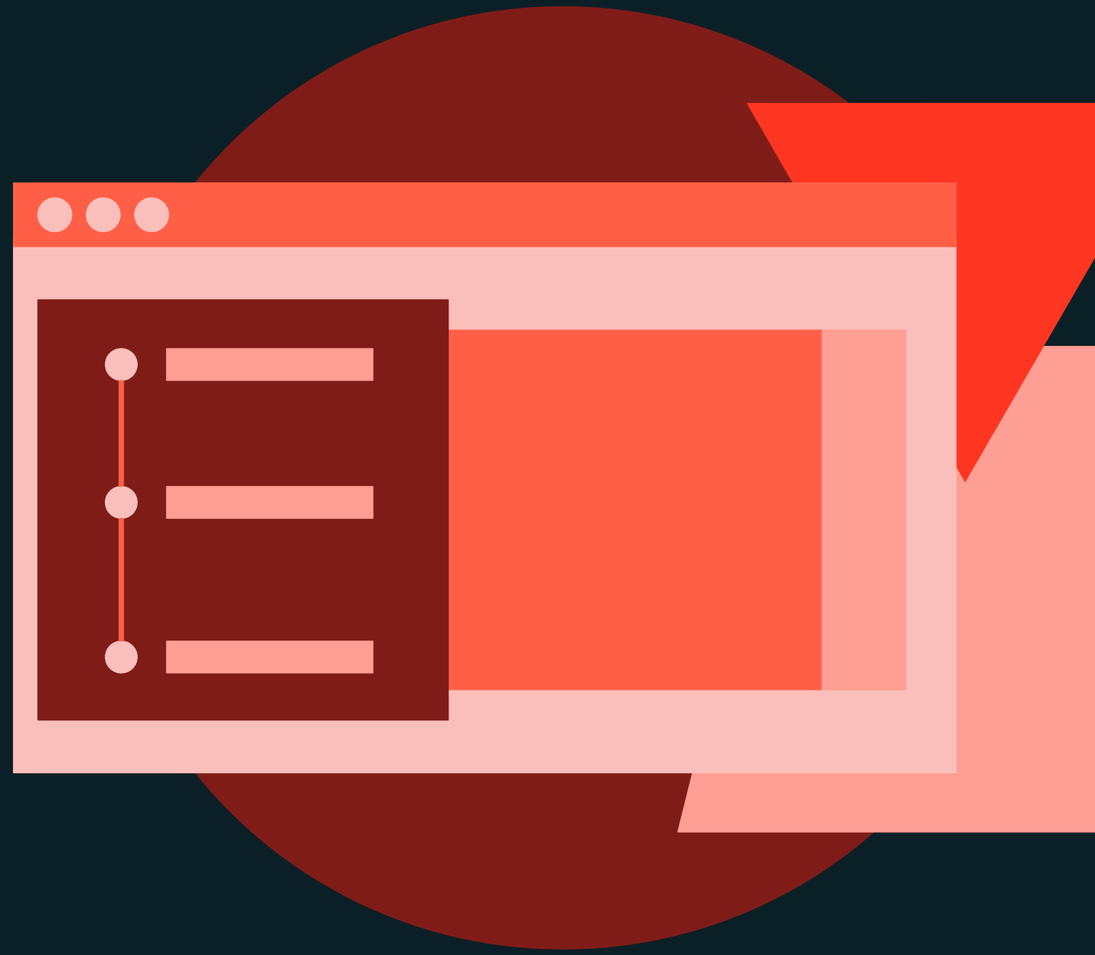


databricks

Architecting MLOps Solutions

DEMONSTRATION

Machine Learning Pipeline Workflow with Databricks API



Demo

Outline

What we'll cover:

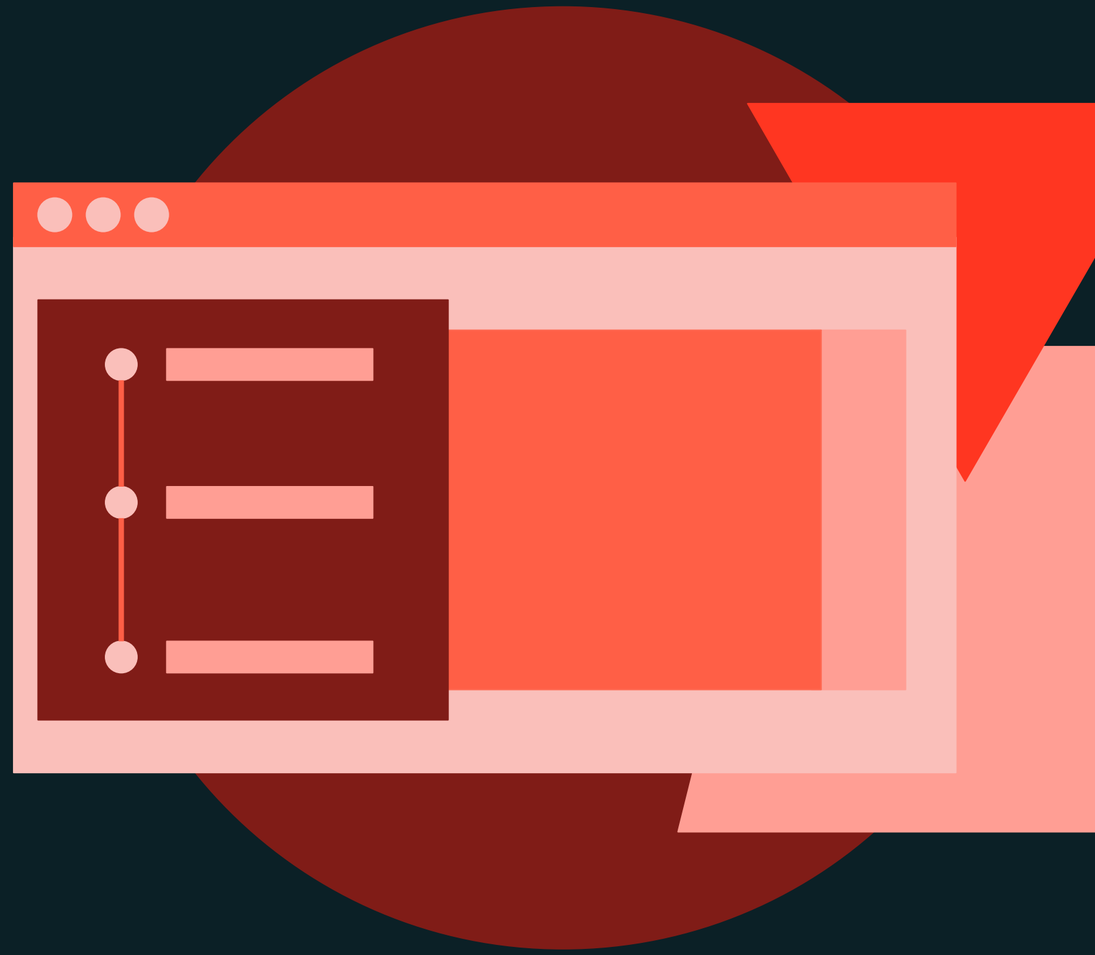
- **Authentication Setup**
  - Configure and authenticate access to the Databricks REST API.
- **Pipeline Configuration**
  - Define and initialize a JSON payload for pipeline tasks.
- **Executing the Pipeline**
  - Trigger the workflow using the Databricks REST API.
- **Monitoring Task Progress**
  - Track job status and task completion using REST API calls.
- **Notifications on Completion**
  - Set up email notifications for workflow completion.
- **Retrieving and Displaying Outputs**
  - Access and visualize JSON data and output files from completed tasks.

databricks

Architecting MLOps Solutions

DEMONSTRATION

Model Testing Job with the Databricks CLI



Demo

Outline

What we'll cover:

- CLI Basics
  - Execute the help command to explore functionalities
- Workflow Job Configuration
  - Create a JSON configuration file for the workflow
- Creating and Running a Workflow Job
  - Create the job using Databricks CLI
  - Extract job ID and run the job
- Monitoring and Exploring Jobs
  - Access the job console
  - View and explore tasks and run output.

databricks

Architecting MLOps Solutions

LAB EXERCISE

Deploying Models with Jobs and the Databricks CLI

Lab

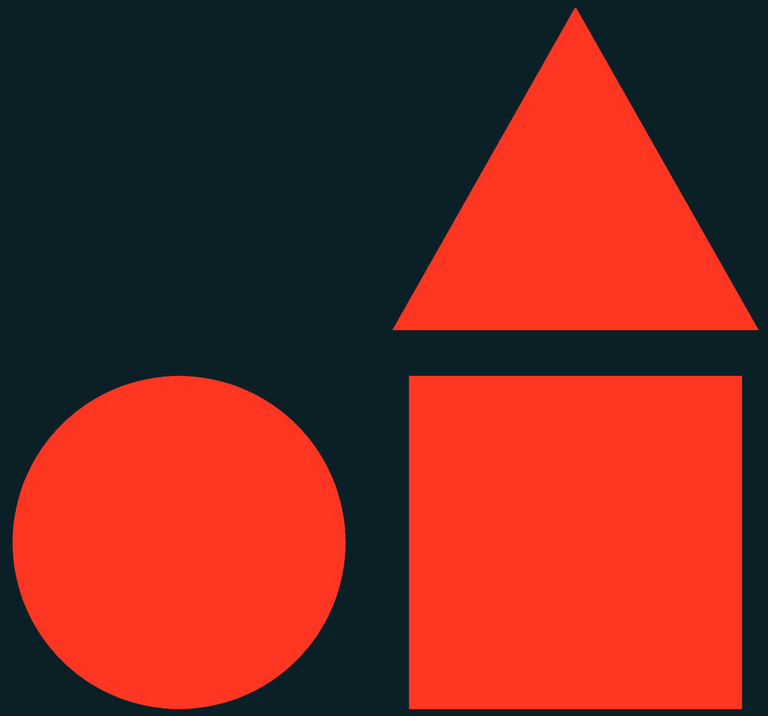
Outline

What you'll do:

- **Task 1:** Identify and update a model's alias to "Champion".
- **Task 2:** Configure and use the Databricks CLI to manage jobs.
- **Task 3:** Create and run a workflow job for model deployment and Batch Inferencing.
- **Task 4:** Monitor and explore the executing workflow job.

databricks

Monitoring MLOps Solutions



Machine Learning Operations

Learning objectives

Things you'll be able to do after completing **this module**

- Develop skills to implement effective model monitoring strategies that encompass business requirements, resource utilization, model performance, and traceability.
- Develop expertise in diagnosing model drift types and setting up appropriate retraining triggers to maintain model accuracy and reliability.
- Develop proficiency in employing monitoring techniques that ensure data integrity, trace model performance, and automate alerts leveraging Databricks' Lakehouse Monitoring.

databricks

Implementation and Monitoring MLOps Solution

LECTURE

Type of Model Monitoring

Type of Model Monitoring

Business Requirements

- Ensuring the ML solution **aligns with and fulfills specific business objectives**.
- Regular assessments to ensure **continued relevance** to evolving business needs.

Model Performance

- Tracking the **accuracy and efficiency** of the model over time.
- Detecting and addressing types of **drift or degradation**.

Resource Utilization

- Ensuring **efficient resource utilization** within the ML infrastructure.
- **Compliance** with Service Level Agreements (**SLAs**) for system performance and availability.

Traceability

- Facilitating **audit trails** for troubleshooting, regulatory compliance, and model improvement.
- **Tracking data lineage** to understand the origin, movement, and transformation of data.

Four Types of Drift

Data Drift:

- Occurs when the **statistical properties** of the input data **change over time**.
- Can impact the model's quality by **introducing inconsistencies** in the data patterns.

Concept Drift:

- Happens when the **relationship** between input features and the **target variable changes**.
- Forces models to adapt to **new patterns** to stay relevant.

Model Quality Drift:

- Reflects a decrease in the model's predictive performance over time.
- Can be **detected through worsening metrics like accuracy, precision, recall, or F1 score.**

Bias Drift:

- Involves shifts in model outcomes that could lead to unfair treatment of certain groups.
- Monitoring for bias drift is **crucial to maintain fairness and ethical standards** in model predictions.

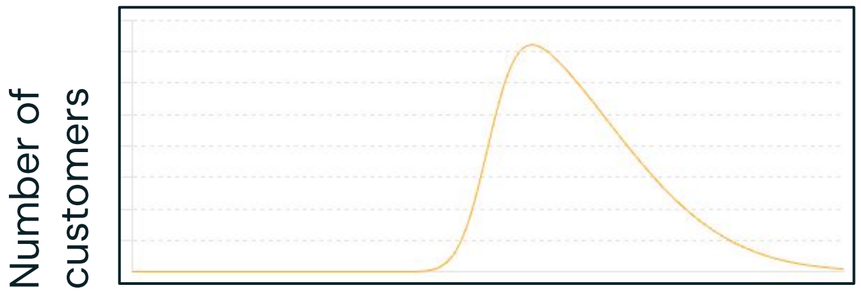
Illustrating Data and Model Drift

Visualizing Changes in Data and Model Performance Over Time

Data Drift

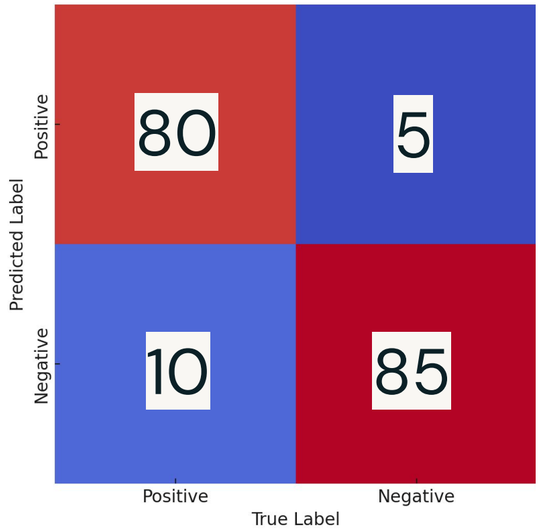
Training data distribution:

Production data distribution:

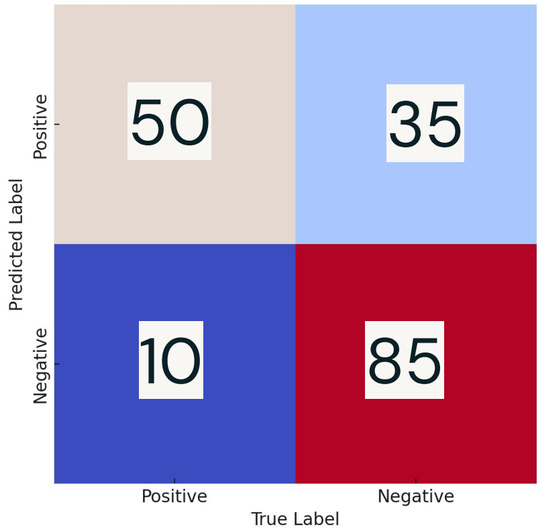


Model Drift

Right After Deployment:



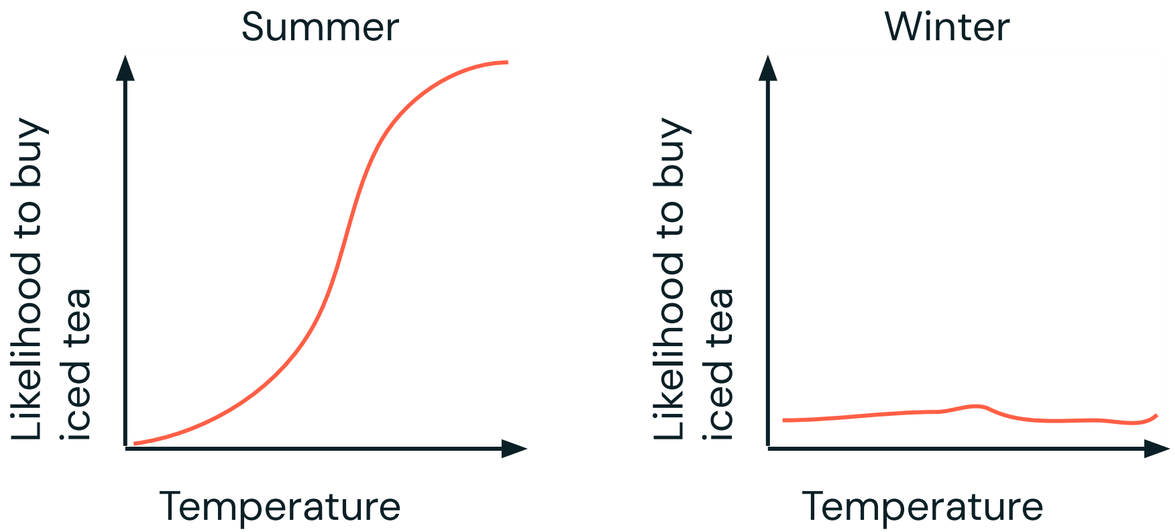
After a Period of Time:



Illustrating Concept and Bias Drift

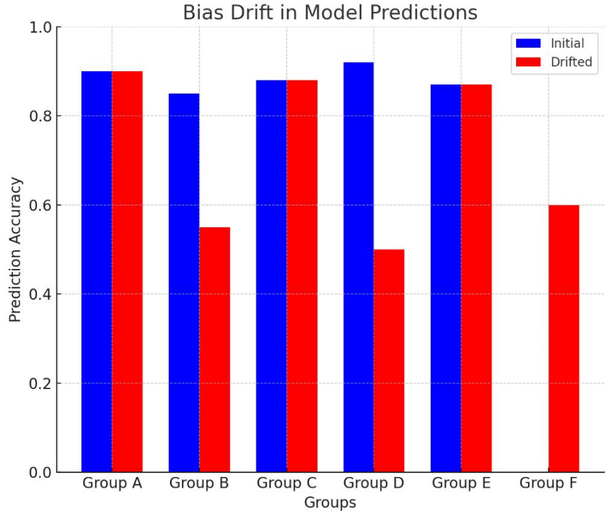
Visualizing Changes in Concept and Bias Over Time

Concept Drift

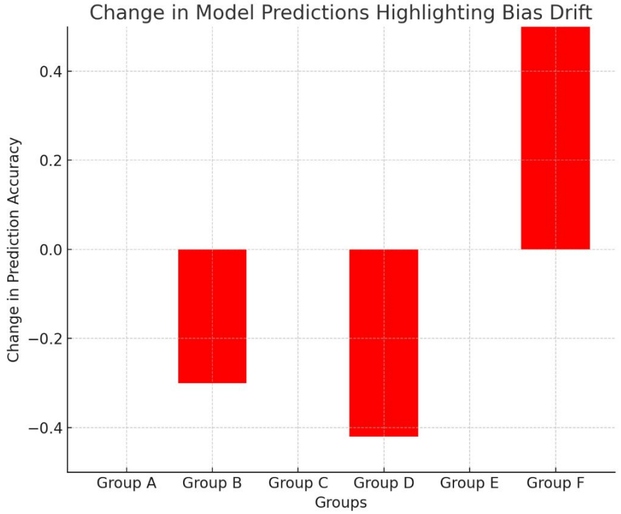


Bias Drift

Change in Group Distribution & New Group



Prediction accuracy varies across different groups



Data Drift Scenario:

Event: Introduction of a New Product Line

Scenario: The company decides to introduce a new line of eco-friendly products, heavily promoting them through social media and email campaigns. This new product line attracts a different demographic compared to the existing customer base.

Changes in Data:

- Demographic Shift
- Browsing Behavior
- Historical Sales Data
- Promotional Activities

Impact of Data Drift:

- **Prediction Accuracy:** Training data no longer represents the current customer behavior and product offerings.
- **Sales Forecasting:** The sales of eco-friendly products is underestimated and the sales of other products is overestimated.

Addressing Data Drift:

- Collect New Data
- Retrain the Model
- Monitor Continuously

Note: The scenario does overlap with concept drift.

ML Model Retraining Triggers

- **Scheduled Retraining:**
  - Databricks recommends starting with scheduled, periodic retraining and moving to triggered retraining when needed.
- **Data Changes:**
  - Changes in the data can either explicitly trigger a retraining job or it can be automated if data drift is detected.
- **Model Code Changes:**
  - Retraining can be triggered by changes in the model code, often due to concept drift or other factors that necessitate an update in the model.
- **Model Configuration Changes:**
  - Alterations in the model configuration can also initiate a retraining job.
- **Monitoring and Alerts:**
  - Jobs can monitor data and model drift, and Databricks SQL dashboards can display status and send alerts

databricks

Implementation and Monitoring MLOps Solution

LECTURE

Monitoring in Machine Learning

Monitoring ML Systems

Continuous logging and review of key component/system metrics

Why?

Used to help diagnose issues before they become **severe** or **costly**

Data to Monitor

Input data (tricky with existing models)
Data in feature stores and vector databases
Human feedback data
Model Outputs

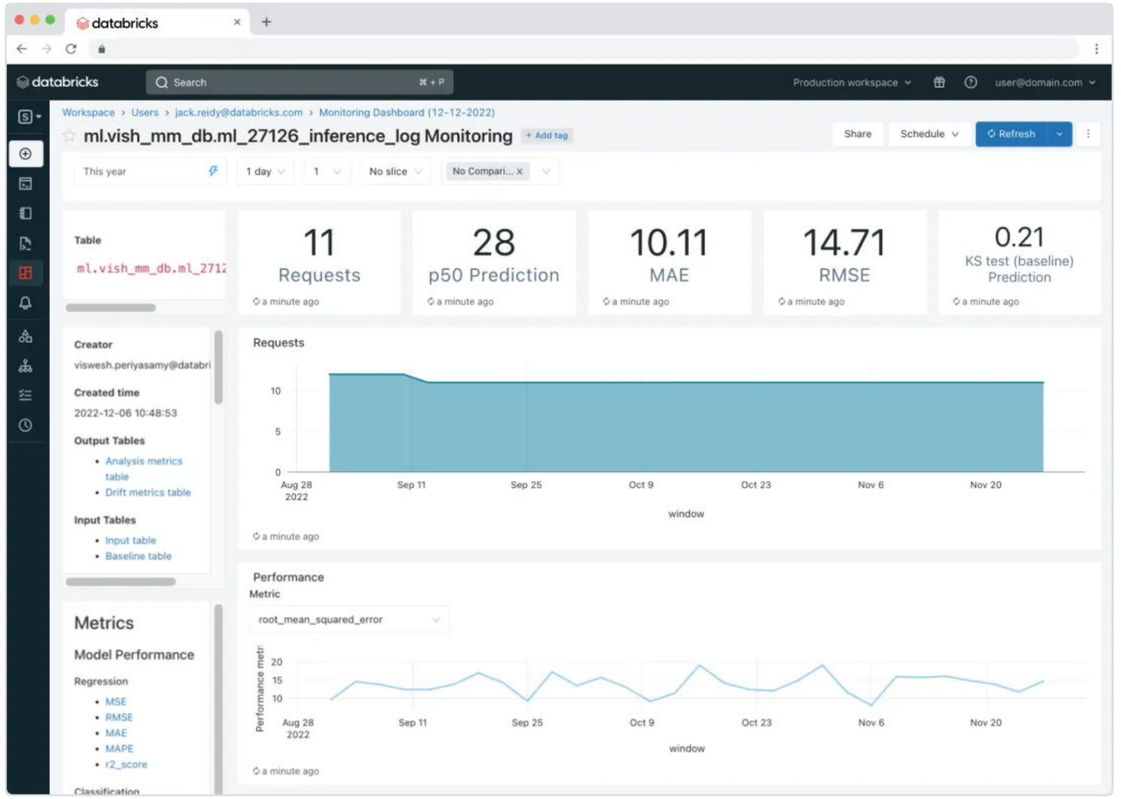
ML Assets to Monitor

Mid-training checkpoints for analysis
Component evaluation metrics
ML system evaluation metrics
Performance/cost details

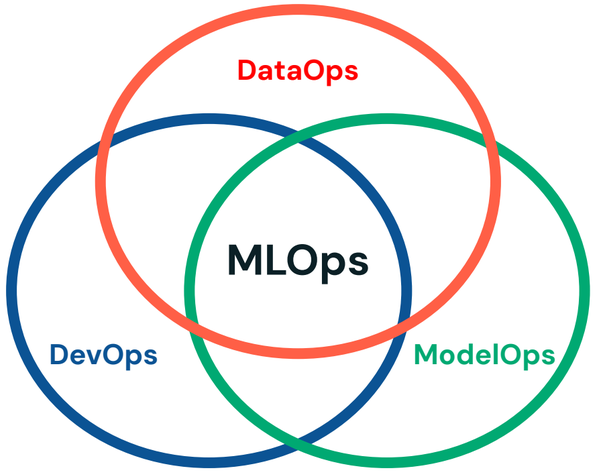
Lakehouse Monitoring

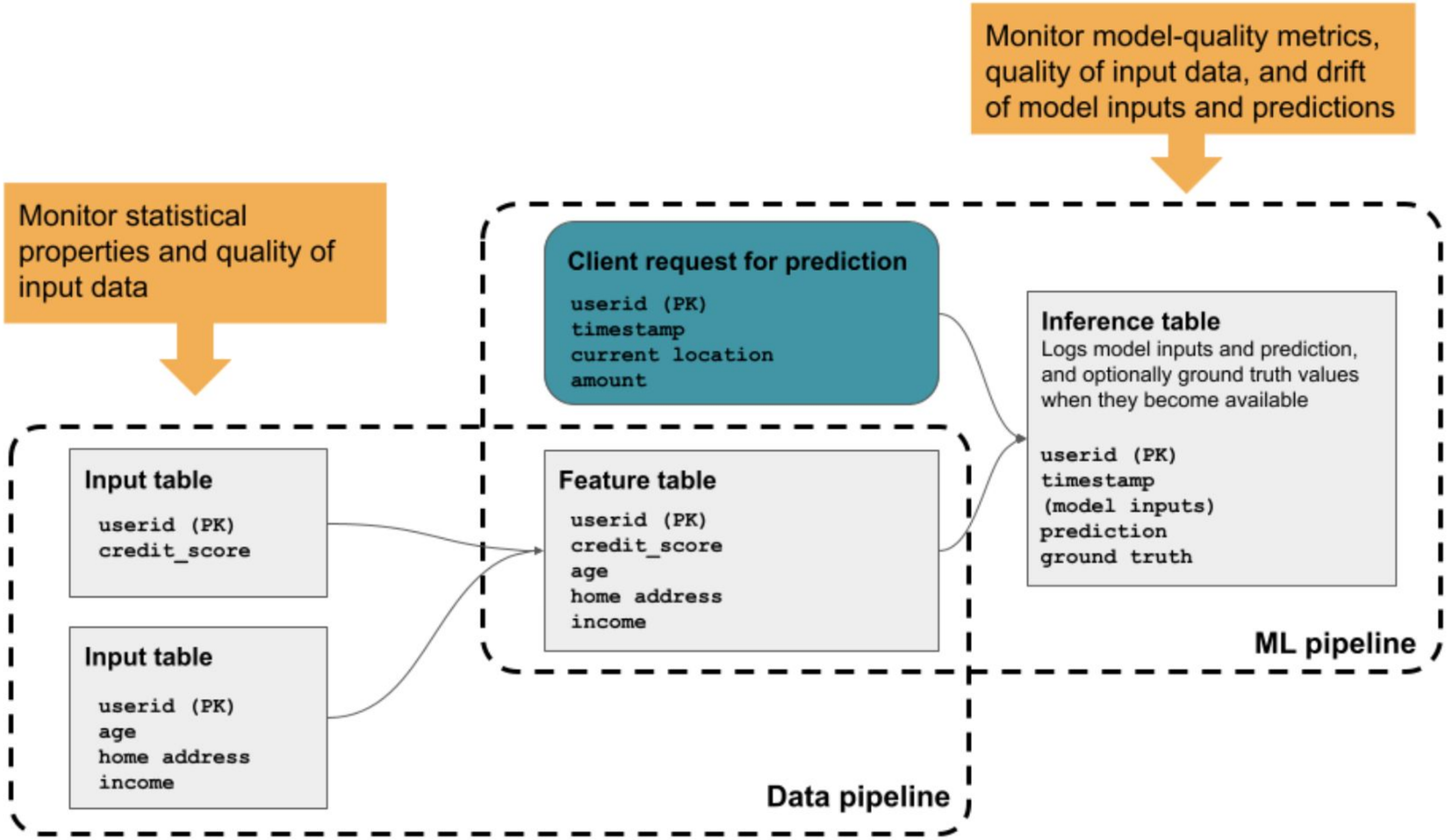
Manage, govern, evaluate, and switch models easily

- Monitor data and AI assets
- Centralized and standardized mechanism for monitoring models in production
- Simplified, built-in tool for monitoring mechanisms to diagnose errors, detect drift, etc.
- Allow for the creation of additional custom metrics.
- Alerting to get notified on drift or quality issues.



What does this look like in practice?





Lakehouse Monitoring Capabilities

Monitor Key Metrics

For fields such as Nulls, null %, zeros, zero %, avg, distincts, distinct %, max, min, stdev, median, max/min/avg length, value frequencies, quantiles, row counts

Define (multiple) time granularities

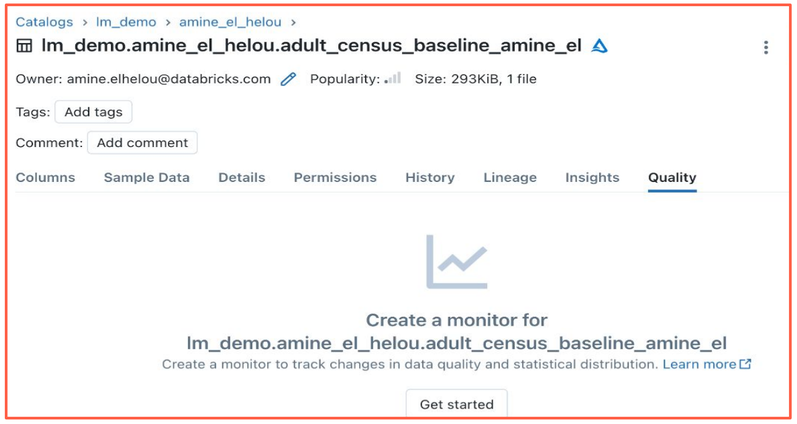
Monitor metrics over time windows, e.g. every day, every 5 minutes, over *n* weeks

Monitor Data Slices

Slice metrics based on columns or predicates, e.g. `state`, `product_class`, `"cart_total > 1000"`

Monitor Tables, VIEWS and ML Models

Consistent quality & drift monitoring of all your production assets including machine-learning model's fairness & bias



UI (UC Data Explorer)

```python
import databricks.lakehouse_monitoring as lm

# Set up monitoring parameters
lm.create_monitor(
  table_name="my_UC_table",
  …)

# Refresh monitoring metrics
lm.run_refresh("my_table")
```

Python API

Lakehouse Monitoring Capabilities

Dashboards and Alerts

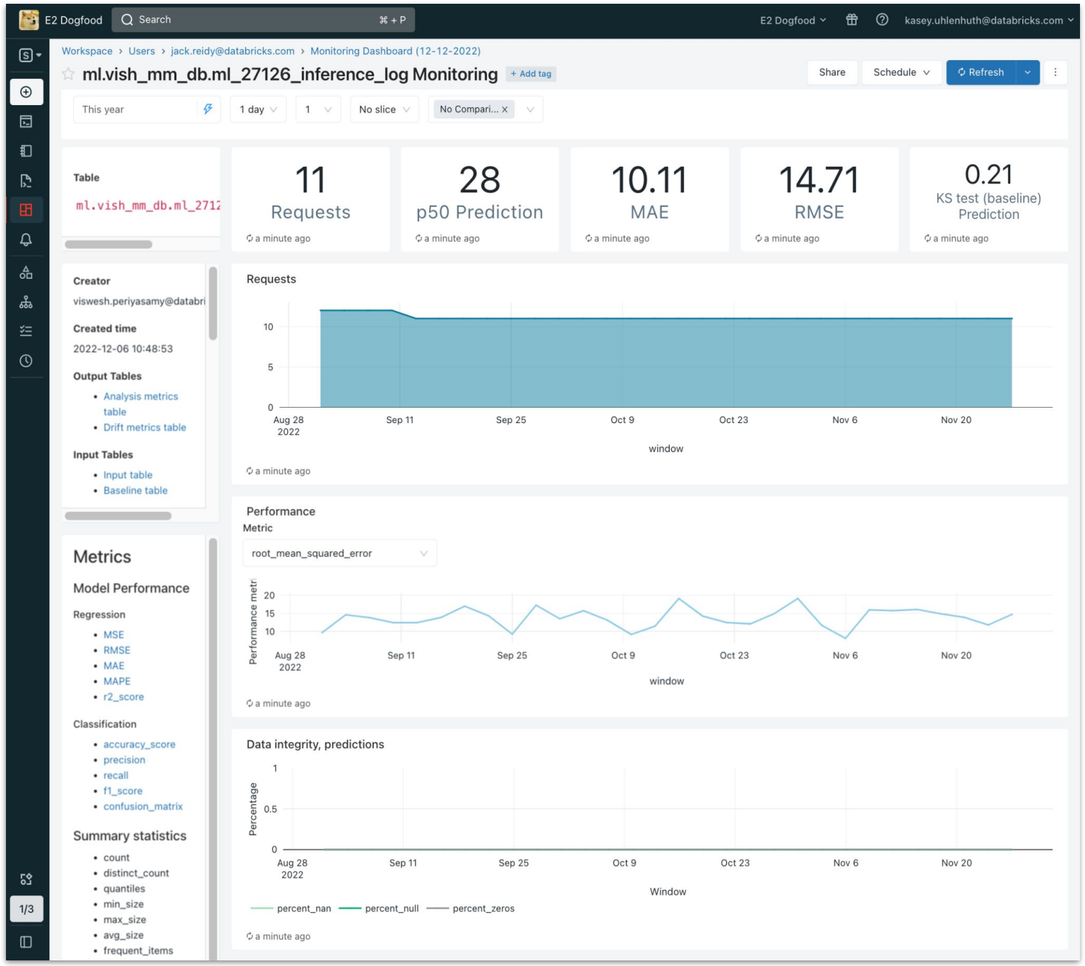
Auto-generated DB SQL dashboards to visualize metrics & trends, SQL Alerts for notifications

Open Monitoring Results

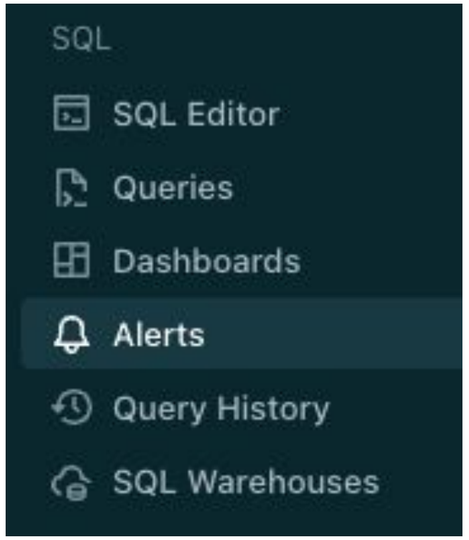
Monitoring results stored in open format Delta tables to build custom analytics using your favorite BI tool

Simple Operations

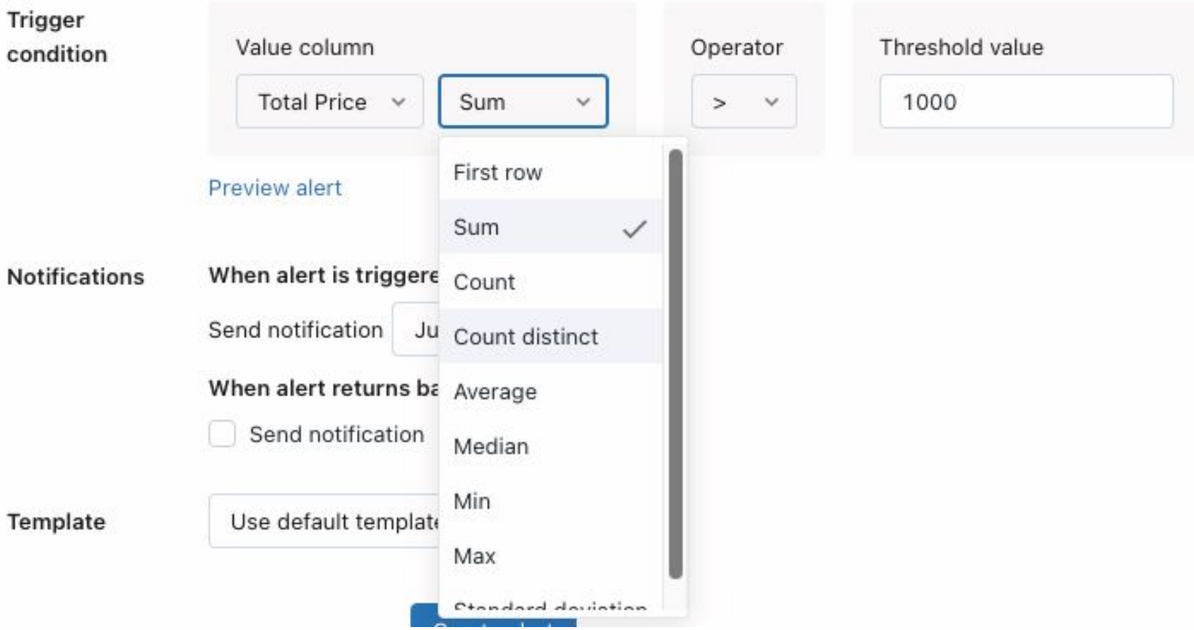
Databricks managed compute eliminates infrastructure management and scaling complexity



What are Databricks SQL alerts?



- Databricks SQL alerts **periodically run** queries, **evaluate defined conditions**, and **send notifications if** a condition is met.
  - **Scheduled Execution:** Automatically runs queries at defined intervals to check specific conditions.
  - **Multi-Channel Notifications:** Receive alerts via Email, Slack, Webhook, MS Teams, PagerDuty, and more.
- **Explore** the documentation for in-depth setup and customization options.

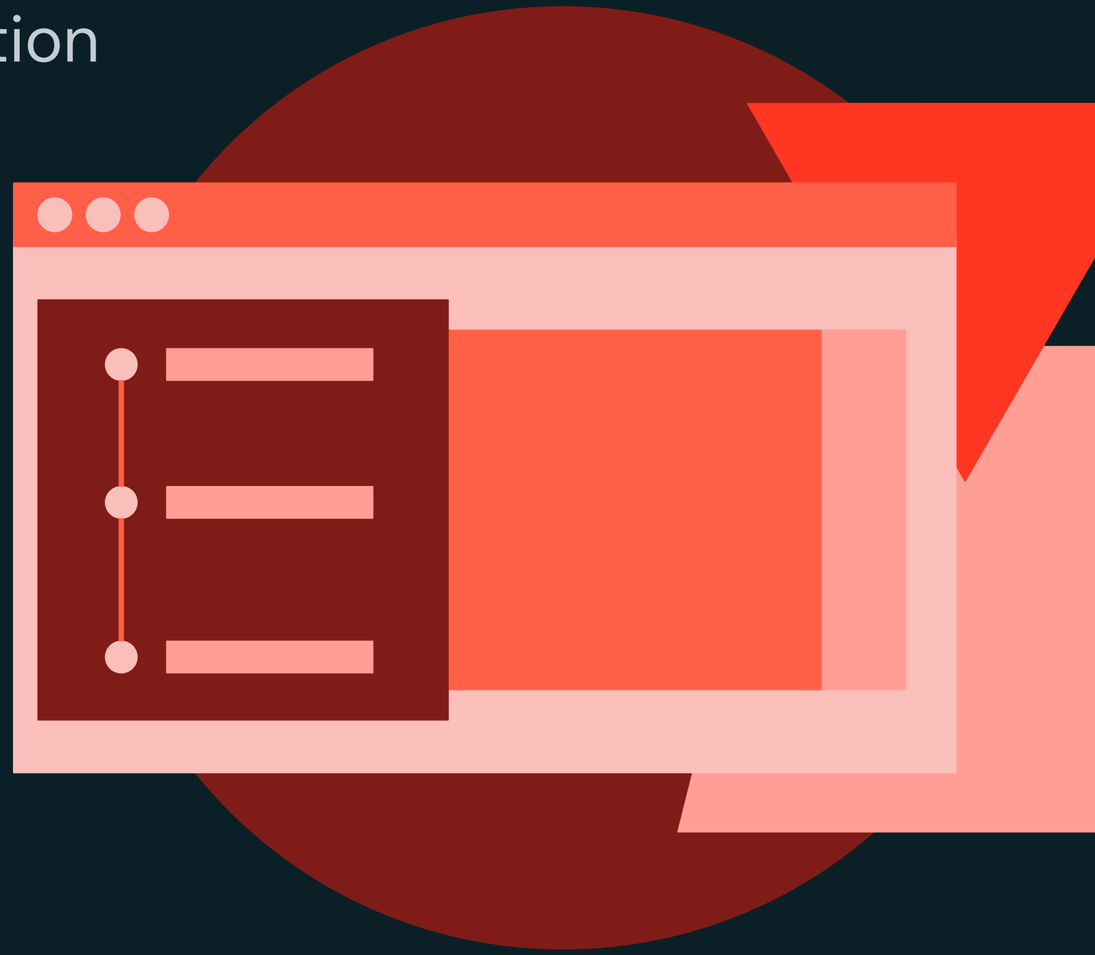


databricks

Implementation and Monitoring MLOps Solution

DEMONSTRATION

Lakehouse Monitoring Dashboard



Demo

Outline

What we'll cover:

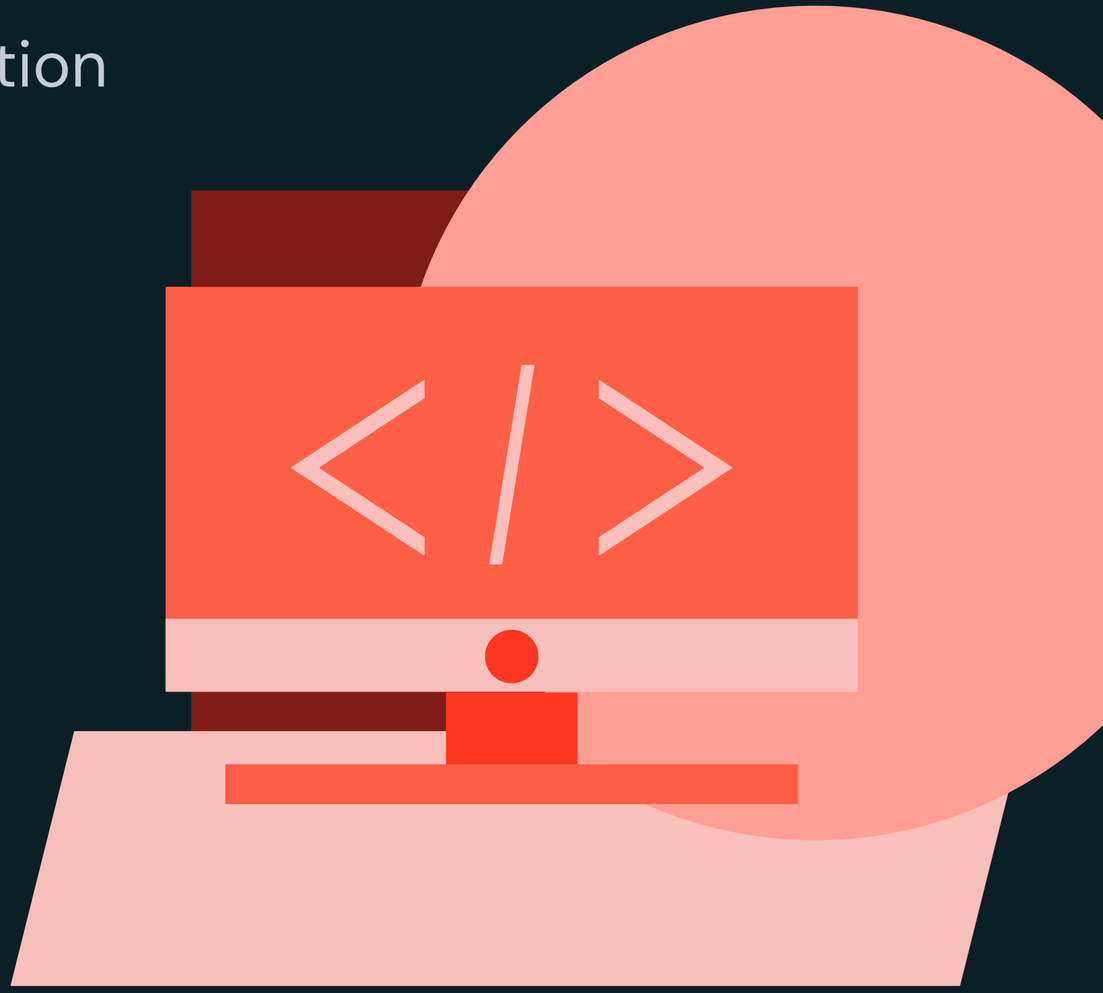
- Train and analyze a machine learning model's inference logs.
- Monitor the model's performance and detect anomalies or drift.
- Handle drift detection and trigger retraining when needed.
- Utilize Databricks Lakehouse Monitoring to continuously track and alert on model performance metrics.

databricks

Implementation and Monitoring MLOps Solution

LAB EXERCISE

Model Monitoring



Lab

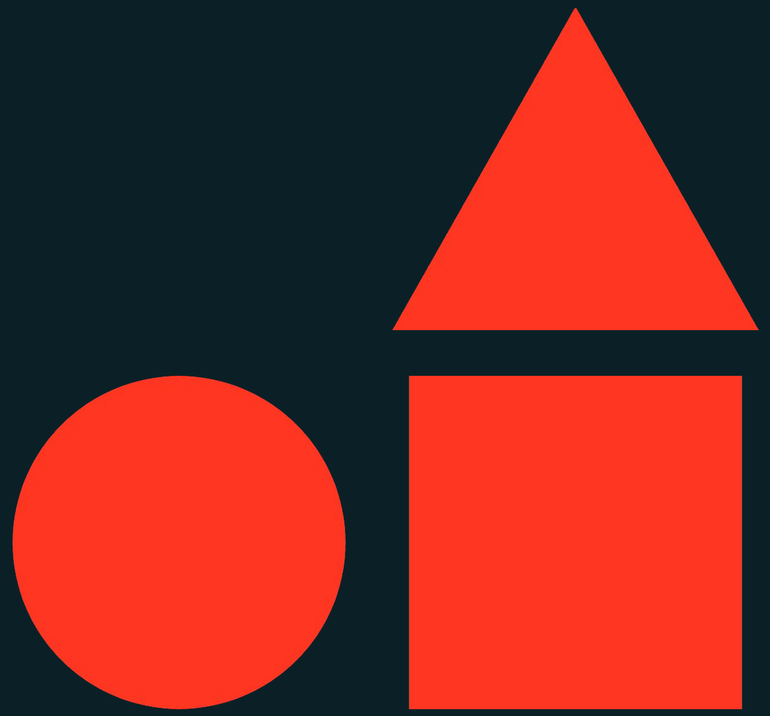
Outline

What you'll do:

- **Task 1:** Save the Training Data as Reference for Drift
- **Task 2:** Processing and Monitoring Inference Data
  - **2.1:** Monitoring the Inference Table
  - **2.2:** Processing Inference Table Data
  - **2.3:** Analyzing Processed Requests
- **Task 3:** Persisting Processed Model Logs
- **Task 4:** Setting Up and Monitoring Inference Data
  - **4.1:** Creating an Inference Monitor with Databricks Lakehouse Monitoring
  - **4.2:** Inspect and Monitor Metrics Tables

databricks

Summary and Next Steps



Machine Learning Operations

databricks